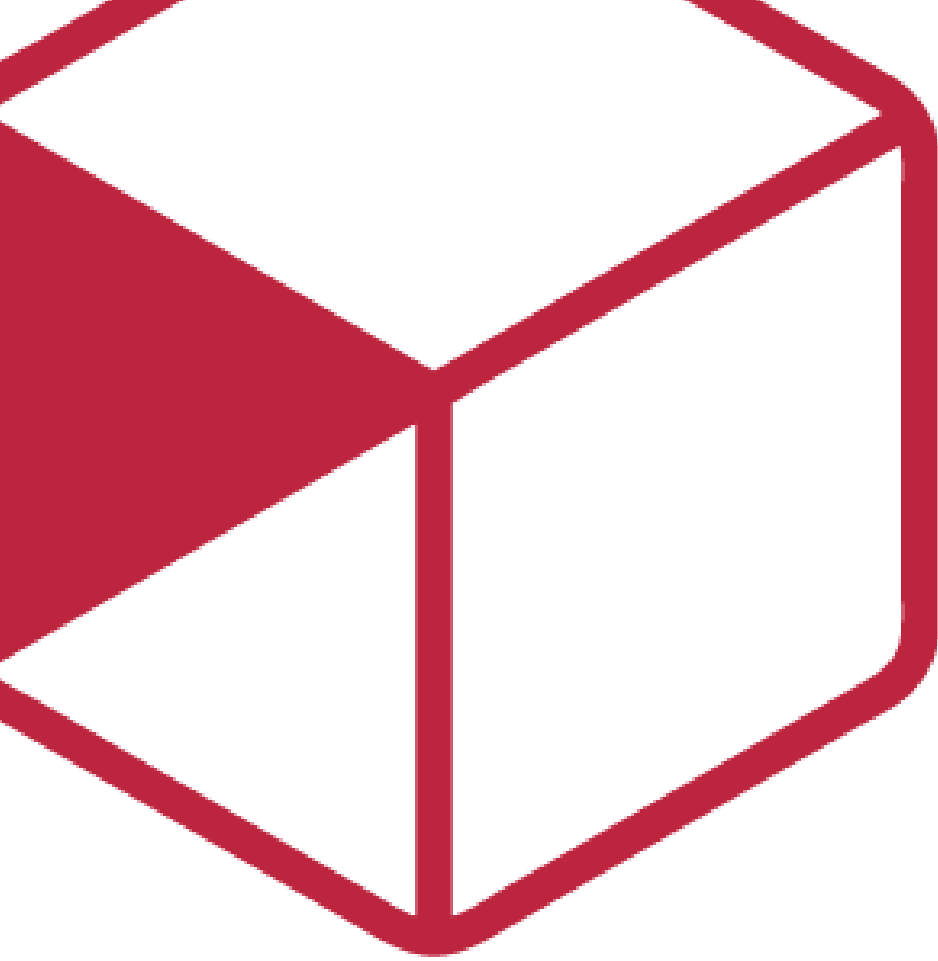




[R16A-RE-FU-008][DIS-SOL]

Diseño de la solución

Requisito TPSC-RE-FU-008 — Buzón de Cobros (Mailbot Rediseñado)

FORMATO	N/A
PROYECTO	R16 - Adquisiciones
REFERENCIA	N/A
VERSIÓN	1.0
FECHA	26 jun 2026
AUTOR	Carlos Iván Morales Carreón
REVISOR	Juan David García Cruz

Contenido

Importante	4
Introducción	4
1. Propósito del documento	4
2. Alcance	4
Específicamente incluye:	4
No se consideran:	5
Visión general del diseño	5
1. Objetivo técnico	5
2. Componentes involucrados	6
Diseño funcional detallado	8
0. Vista de alto nivel — Entrada → Proceso → Salida	9
1. Flujo principal — Recepción y clasificación de correo	12
Detalle — Idempotencia ante push duplicado (paso 6)	13
Detalle — Entidades insertadas al persistir el correo (paso 12)	13
2. Flujo — Consulta del Buzón de Cobros	15
3. Flujo — Reclasificación manual	15
4. Flujo — Cierre y eliminación del pendiente	17
5. Flujo — Bandeja del Coordinador de Tesorería (OBS-021)	18
6. Criterios de aceptación del requisito	20
7. Reglas técnicas aplicadas	22
Diseño de componentes	26
1. Diagramas	26
Flujo end-to-end (Gmail Push → Buzón del Gestor)	26
Arquitectura de la solución MailbotWorker.sln	28
Infraestructura Google Cloud Pub/Sub	29
Estrategia por ambiente — Pull (Desarrollo) vs Push (Staging/Prod)	30
Ciclo de vida del pendiente en el Buzón	31
Bandeja del Coordinador de Tesorería (OBS-021)	32
2. Contratos de endpoints	33
POST /BuzonCobros	33
PUT /api/buzon/cobros/{id}/reclasificar	34
PUT /api/cobros/folio/{id}/cerrar	34
POST /BandejaCoordinadorTesoreria	34
PUT /bandeja/coordinador/{id}/reclasificar	35
3. Interfaces del dominio	35
Contrato JSON del Agente IA	36
4. Query SQL — Buzón de Cobros	37
5. Endpoints internos de Mailbot.Api	38
POST /webhook/gmail/{region}	39



GET /health	39
GET /metrics	39
GET /admin/mailbot/eventos	40
POST /admin/mailbot/eventos/{id}/retry	40
POST /admin/mailbot/eventos/{id}/descartar	41
Impacto Técnico	42
1. Impacto en código existente	42
2. Impacto en modelos por actualización de modelos	44
ProquifaDotNet — BD ProquifaDotNet	44
PConnectProquifaDotNet — BD existente, reutilizada por LegacyBridge	46
3. ProquifaDotNet.LegacyBridge — impacto detallado	46
Diseño — Buzones de Cotizaciones y Pedidos (T25-T32)	51
Diferencias frente al Buzón de Cobros	52
Flujo — Consulta y reclasificación (ambos buzones, mismo patrón)	53
Contratos de endpoints	53
Corte de producción del Mailbot legacy (decisión D5)	54
Manejo de Errores y Excepciones	54
Estrategia de Pruebas (Diseño de las pruebas)	56
1. Pruebas funcionales (Criterios de Aceptación en DEV)	56
2. Pruebas técnicas (unitarias e integración)	57
a. Unitarias	57
b. Pruebas de integración	57
3. Casos críticos	57
Control de versiones	59



Importante

Posterior a este diseño, ¿Cómo saber si el diseño de la solución al requisito está completo para que el programador inicie con el desarrollo?

Hazte estas preguntas rápidas:

- ¿El programador sabe qué flujo implementar?
- ¿Sabe qué pasa si algo falla?
- ¿Sabe qué reglas no puede romper?
- ¿Sabe qué pruebas debe pasar?
- ¿Sabe dónde impacta?

Nota: Este documento es una propuesta basada en el estándar IEEE 1016 "Software Design Description". Se debe administrar el tiempo de diseño para completarlo considerando todos los detalles técnicos.

Introducción

1. Propósito del documento

El propósito de este documento es definir el diseño de la solución técnica para el requisito **TPSC-RE-FU-008 — Buzón de Cobros (Propuesta 2)**, describiendo la arquitectura Gmail Push Notifications + Google Cloud Pub/Sub + Worker .NET 10 que reemplaza el Mailbot legacy, los componentes de MailbotWorker.sln, la lógica de buzón en ProquifaDotNet, la generación automática de pendientes en Validar Cobro y la integración con el Agente IA.

Nota: Este documento se enfoca en el diseño técnico de la solución. No redefine las reglas de negocio del requisito ni la lógica de captura que ocurre en Validar Cobro (FU-023).

2. Alcance

Este análisis cubre únicamente el requisito TPSC-RE-FU-008 y considera:

- Componentes involucrados en las tres soluciones (ProquifaDotNet, MailbotWorker.sln, ProquifaDotNet.LegacyBridge)
- Flujo técnico end-to-end desde Gmail push hasta la bandeja del Gestor
- Contratos de servicios e interfaces entre capas
- Integración con Gmail API, Google Cloud Pub/Sub y Agente IA
- Ajustes en ProquifaDotNet (.NET 4.8) para soportar el Buzón de Cobros
- Impacto en base de datos y modelos existentes
- Manejo de errores, backoff y DLQ
- Estrategia de pruebas por solución

Específicamente incluye:

- Nueva solución MailbotWorker.sln (.NET 10): Api, Worker, Application, Domain, Infrastructure, Tests
- Gmail API watch por región (MEX / PER) con renovación automática cada 6 días
- Google Cloud Pub/Sub como canal de entrega (Topics + Subscriptions Push + Dead Letter Topic)
- Agente IA (Gemini Flash-Lite vía Vertex AI — P5 resuelta 2026-06-19) para clasificación y extracción de datos del correo y adjuntos
- Buzón de Cobros en ProquifaDotNet: lista paginada filtrada por gestor autenticado y región
- Buzones de Cotizaciones y Pedidos recreados con el mismo patrón (empezando por Cobros)
- Generación automática de pendiente en Validar Cobro (fccFolioPagoCliente) al clasificar como cobro
- Reclasificación manual: mover correo a otro buzón
- Cierre/eliminación automática del pendiente por eventos en Validar Cobro
- Bandeja del Coordinador de Tesorería (OBS-021): correos de cobro no enrutables (Caso 1 y Caso 2)
- MinIO para almacenamiento de adjuntos; Brevo para alertas de error al equipo operativo
- Trazabilidad: tablas MailbotEventoCorreo y MailbotClasificacionLog
- Infraestructura base ProquifaDotNet.LegacyBridge (.NET 10) + Canal ETLcobros (Buzón de Cobros → Legacy PConnect)

No se consideran:

- Captura de datos del cobro — ocurre en Validar Cobro (FU-023)
- Eliminación directa del correo por el Gestor — el ESAC la realiza desde el buzón Otros
- Criterios configurables de clasificación en la interfaz — los prompts .txt son el mecanismo de ajuste
- Diseño definitivo del canal ETLcobros — el molde de tabla y el mecanismo ya están resueltos para diseño; el valor exacto de DoctosR en la instancia legacy RYNL015 es el único detalle que sigue abierto (Tarea ETLcobros-T10)
- **Sin pendientes de Alcance abiertos.** *Los 3 pendientes listados en v2.0 quedaron resueltos: el proveedor de IA (2026-06-19, Gemini Flash-Lite vía Vertex AI — ver DAR-Proveedor-IA v1.1), los prompts del Agente IA (2026-06-19, muestra de 8 correos reales con datos extraídos directo de PConnect.dbo.DoctosR) y el alcance del rol "Gerente de Tesorería" en la Bandeja del Coordinador (2026-06-19, decisión de Carlos por simetría con OBS-003 de FU-002 — ver RT-12).*

Visión general del diseño

1. Objetivo técnico



"Reemplazar el Mailbot legacy (.NET 4.8 + polling + ML.NET) por una arquitectura orientada a eventos usando Gmail Push Notifications y Google Cloud Pub/Sub, garantizando clasificación con Agente IA en segundos, trazabilidad completa de eventos, idempotencia ante reintentos de Pub/Sub, y generación automática del pendiente de cobro en Validar Cobro sin intervención manual."

2. Componentes involucrados

Aplicativo	Componente	Responsabilidad	Ubicación
GCP	Gmail API watch	Notifica a Pub/Sub cuando llega un correo nuevo al INBOX	Google Cloud Platform — proquifa-prod
GCP	Google Cloud Pub/Sub	Entrega el push HTTP a Mailbot.Api por región	Topics: mailbot-mex / mailbot-per / mailbot-dlq
MailbotWorker.sln	Mailbot.Api (Minimal API .NET 10)	Recibe el push de Pub/Sub, valida token, persiste MailbotEventoCorreo y encola en Channel interno	Mailbot.Api/Endpoints/WebhookGmailEndpoints.cs
MailbotWorker.sln	IEventoChannel / EventoCorreoChannel	Desacopla la recepción HTTP del procesamiento del Worker	Mailbot.Infrastructure/Channels/EventoCorreoChannel.cs
MailbotWorker.sln	CorreoWorkerMex / CorreoWorkerPer	Consume el Channel, obtiene el correo completo de Gmail API y orquesta el procesamiento	Mailbot.Worker/Workers/
MailbotWorker.sln	ProcesarCorreoUseCase	Orquesta clasificación IA, persistencia BD y guardado de adjuntos	Mailbot.Application/UseCases/
MailbotWorker.sln	GeminiClasificadorA gente	Clasifica el correo y extrae datos según	Mailbot.Infrastructure/IA/

		tipo (cobro / pedido / cotización / otros)	
MailbotWorker.sln	GmailWatchRenovacionWorker	Renueva el watch de Gmail cada 6 días automáticamente	Mailbot.Worker/Workers/
MailbotWorker.sln	GmailPubSubPullListener (solo Desarrollo)	Consume la subscription Pull vía SubscriberClient y publica en IEventoChannel — evita exponer un endpoint HTTPS público en dev (local o servidor DEV). En Staging/Prod no se usa: Mailbot.Api recibe el push directamente (RT-16)	Mailbot.Worker/Workers/GmailPubSubPullListener.cs
MailbotWorker.sln	MinioService	Guarda adjuntos del correo en bucket mailbot/	Mailbot.Infraestructure/MinIO/
MailbotWorker.sln	BrevoNotificacionService	Envía alertas de error al equipo de operaciones	Mailbot.Infraestructure/Brevo/
ProquifaDotNet (.NET 4.8)	BuzonCobrosB0	Lista paginada del Buzón de Cobros filtrada por IdUsuarioCobrador e IdRegion	Logic.Pqf.Logistica/L11.MailBot/
ProquifaDotNet (.NET 4.8)	BuzonCobrosController	Expone POST /BuzonCobros — cobrador y región extraídos del token JWT	WebApi.Logistica/Controllers/Procesos/Mailbot/
ProquifaDotNet (.NET 4.8)	FolioPagoClienteB0 (modificación)	Genera y cierra/elimina fccFolioPagoCliente (pendiente en Validar Cobro) — no es	Logic.Pqf.Logistica/L11.MailBot/Procesos/Pagos/CorreoRecibidoClienteToPagoB0.cs



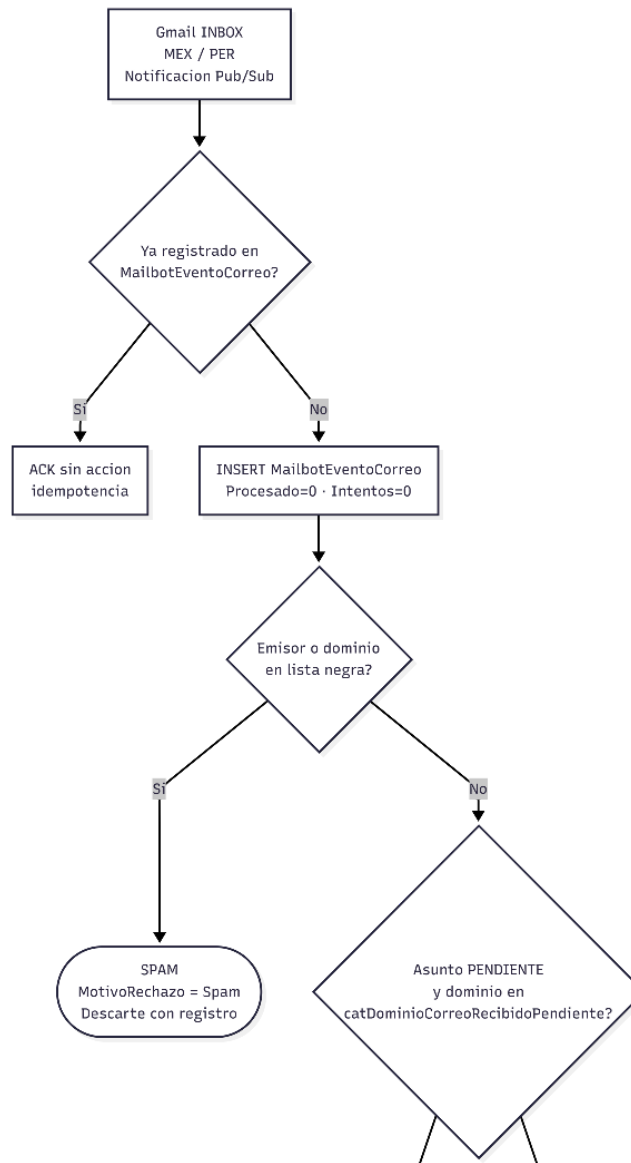
		archivo nuevo, es lógica agregada a CorreoRecibidoClienteToPagoB0.cs existente	
BD SQL Server	ProquifaDotNet	Almacena correos, clasificaciones, pendientes y trazabilidad de eventos	SQL Server — tablas existentes + nuevas

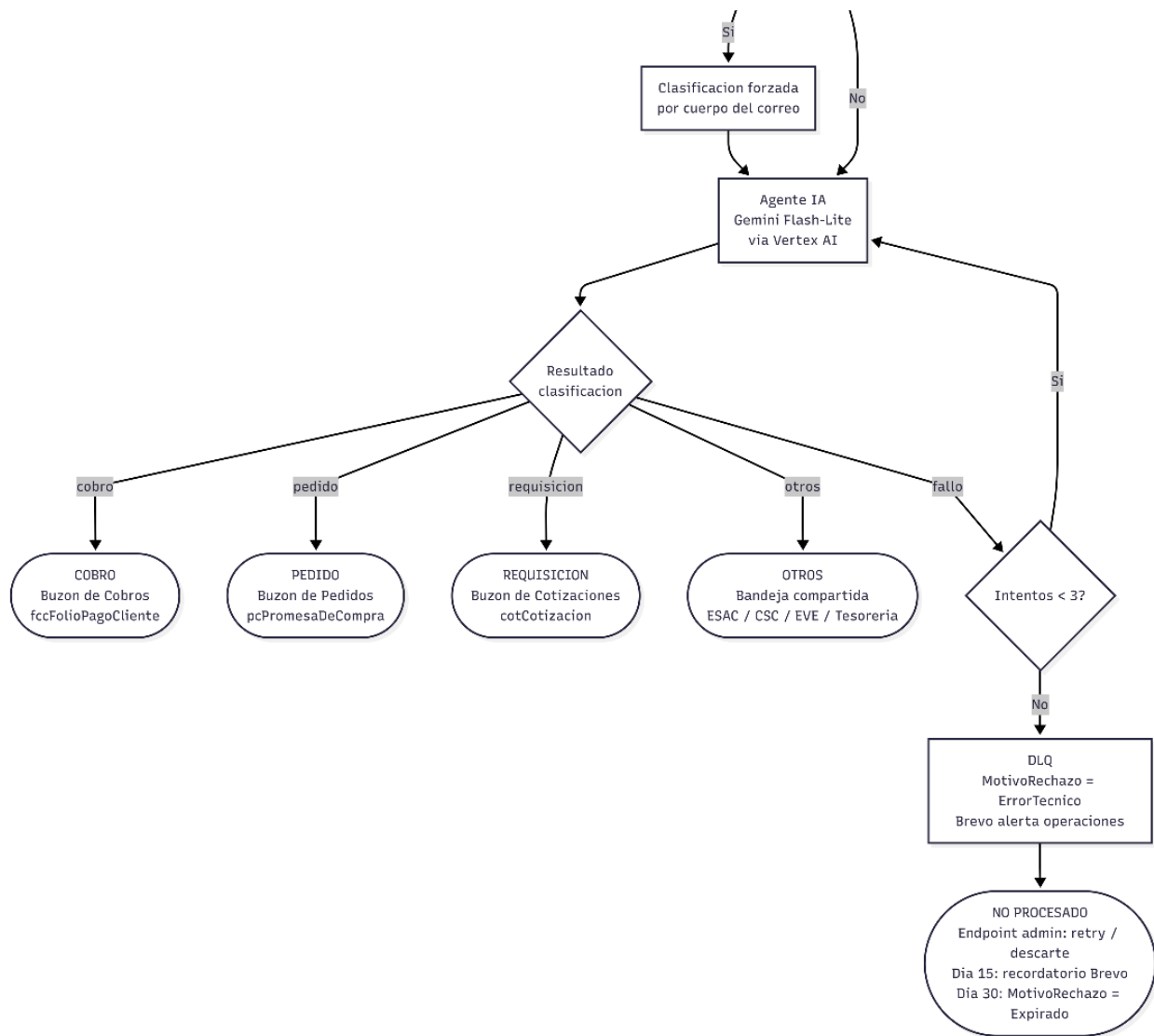
Diseño funcional detallado

Descripción paso a paso de los 5 flujos identificados para este requisito, en orden lógico.

0. Vista de alto nivel — Entrada → Proceso → Salida

Regla de oro: todo correo que entra al sistema tiene exactamente un estado final. No existe correo sin estado final asignado.





None

flowchart TD

INB["Gmail INBOX
MEX / PER
Notificacion Pub/Sub"]

IDEM{"Ya registrado en
MailbotEventoCorreo?"}

ACK["ACK sin accion
idempotencia"]

REG["INSERT MailbotEventoCorreo
Procesado=0 · Intentos=0"]

SPAM_CHK{"Emisor o dominio
en lista negra?"}

SPAM_FIN(["SPAM
MotivoRechazo = Spam
Descarte con registro"])



```

    PEND_CHK{"Asunto PENDIENTE<br/>y dominio
en<br/>catDominioCorreoRecibidoPendiente?"}
    PEND["Clasificacion forzada<br/>por cuerpo del correo"]

    IA["Agente IA<br/>Gemini Flash-Lite<br/>via Vertex AI"]

    CLASIF{"Resultado<br/>clasificacion"}

    COBRO(["COBRO<br/>Buzon de Cobros<br/>fccFolioPagoCliente"])
    PEDIDO(["PEDIDO<br/>Buzon de Pedidos<br/>pcPromesaDeCompra"])
    REQ(["REQUISICION<br/>Buzon de Cotizaciones<br/>cotCotizacion"])
    OTROS(["OTROS<br/>Bandeja compartida<br/>ESAC / CSC / EVE / Tesoreria"])

    RETRY{"Intentos < 3?"}
    DLQ["DLQ<br/>MotivoRechazo = ErrorTecnico<br/>Brevo alerta operaciones"]
    NOPROC(["NO PROCESADO<br/>Endpoint admin: retry / descarte<br/>Dia 15:
recordatorio Brevo<br/>Dia 30: MotivoRechazo = Expirado"])

    INB --> IDEM
    IDEM -- Si --> ACK
    IDEM -- No --> REG
    REG --> SPAM_CHK
    SPAM_CHK -- Si --> SPAM_FIN
    SPAM_CHK -- No --> PEND_CHK
    PEND_CHK -- Si --> PEND --> IA
    PEND_CHK -- No --> IA
    IA --> CLASIF
    CLASIF -- cobro --> COBRO
    CLASIF -- pedido --> PEDIDO
    CLASIF -- requisicion --> REQ
    CLASIF -- otros --> OTROS
    CLASIF -- fallo --> RETRY
    RETRY -- Si --> IA
    RETRY -- No --> DLQ --> NOPROC

```

Estados finales del sistema:

Estado	Destino	Visible para
COBRO	Buzon de Cobros	AnalistaDeCuentasPorCobrar (Gestor de Cobranza)
PEDIDO	Buzon de Pedidos	ESAC, CoordinadorDeServicioAlClien te, EVI

REQUISICION	Buzon de Cotizaciones	EVI, EVE
OTROS	Bandeja compartida	ESAC, CoordinadorDeServicioAlClien nte, EVE, AnalistaDeCuentasPorCobrar (RT-18)
SPAM	Descarte con registro	Nadie — MotivoRechazo = Spam en MailbotEventoCorreo
NO PROCESADO	BD tecnica + Brevo	Equipo de operaciones via alerta. Acciones via endpoint admin

Nota OTROS: `catClasificacionCorreoRecibido WHERE Clave = 'otros'` tiene `AnalistaDeCuentasPorCobrar = 0` en la BD actual. RT-18 define el UPDATE requerido para que Tesorería vea estos correos. Efecto colateral conocido y aceptado: los correos otros de cotizaciones y pedidos también aparecerán en la bandeja de Tesorería.

1. Flujo principal — Recepción y clasificación de correo

- Gmail recibe un correo en el INBOX MEX o PER.
- Gmail notifica a Google Cloud Pub/Sub (Topic mailbot-mex o mailbot-per) con un push que incluye { emailAddress, historyId }.
- Pub/Sub realiza un POST HTTP a Mailbot.Api en /webhook/gmail/{region}.
- PubSubTokenValidationMiddleware valida el Bearer token de Google; rechaza con 401 si es inválido.
- Mailbot.Api decodifica el payload base64 y extrae { emailAddress, historyId }.
- Mailbot.Api hace INSERT en MailbotEventoCorreo CON Procesado = 0, Intentos = 0 (idempotencia por IdentificadorCorreoGmail).
- Mailbot.Api publica un EventoCorreoDto en el IEventoChannel (Channel interno).
- Mailbot.Api responde 200 OK → Pub/Sub hace ACK y elimina el mensaje de la cola.
- CorreoWorkerMex o CorreoWorkerPer consume el EventoCorreoDto del Channel.
- ProcesarCorreoUseCase llama a GmailService.ObtenerCorreo(historyId) → recibe correo completo (asunto, cuerpo, adjuntos).
- GeminiClasificadorAgente.Clasificar(correo) → retorna { clasificacion, confianza, datosExtraidos }.
- INSERT en CorreoRecibido + CorreoRecibidoCliente + ArchivoCorreoRecibido.
- INSERT en MailbotClasificacionLog (ClasificacionIA, Confianza, ModeloVersion, Tokens) — camino exitoso del Agente IA. Ver RT-21 para el registro del camino de fallo.
- Si clasificacion == 'cobro': GenerarPendienteUseCase → INSERT fccFolioPagoCliente con campos *MailBot pre-cargados desde datosExtraidos del Agente IA, obtenidos con el prompt extraccion_cobro.txt. Inmediatamente después,

GenerarPendienteUseCase notifica a ProquifaDotNet.LegacyBridge (POST /sync/registrar, {NombreEntidad:"BuzonCobros", IdentificadorRegistro:IdFolioPagoCliente}) para disparar el Canal ETL Cobros — ver sección "3. ProquifaDotNet.LegacyBridge", subsección "Mecanismo de detección y disparo del ETL (P20)". Si la llamada falla, no se reintenta en caliente aquí: el barrido recurrente de LegacyBridge.Worker lo detecta después.

15. Si hay adjuntos: MinioService.GuardarArchivo(adjunto) en bucket mailbot/.
16. UPDATE MailbotEventoCorreo SET Procesado = 1, FechaProcesado = GETDATE().
17. Si error en cualquier paso → backoff exponencial (5s / 10s / 20s) → NACK → Pub/Sub DLQ (mailbot-dlq). Si el error ocurrió en el paso 11 (el Agente IA falló antes de devolver resultado): INSERT en MailbotClasificacionLog con ClasificacionIA = NULL, ErrorIA = <mensaje>, TipoFallo = <timeout|parse-error|rate-limit|error-generic> (RT-21) — ocurre en el bloque catch del Worker, antes del NACK.

Detalle — Idempotencia ante push duplicado (paso 6)

Pub/Sub garantiza *at-least-once delivery* — el mismo push puede llegar más de una vez (reintento de red, ack tardío, etc.). Paso a paso:

1. Mailbot.Api recibe el push y decodifica { emailAddress, historyId }.
2. Antes de insertar, ejecuta ICorreoRepository.ExisteEventoAsync(identificadorCorreoGmail, idRegion) — identificadorCorreoGmail se construye como {emailAddress}:{historyId} (ver Regla RT-02).
3. **Si no existe:** INSERT en MailbotEventoCorreo (Procesado = 0), publica en el IEventoChannel, responde 200 OK.
4. **Si ya existe:** no se inserta de nuevo ni se publica en el Channel — el correo ya está en proceso o ya fue procesado. Responde 200 OK igualmente (devolver cualquier código fuera de 2xx hace que Pub/Sub reintente indefinidamente un evento que ya es idempotente).
5. Caso límite — dos pushes casi simultáneos pasan el paso 2 antes de que el primero termine el INSERT: lo resuelve el índice UNIQUE NONCLUSTERED INDEX UIX_MailbotEventoCorreo_Gmail (IdentificadorCorreoGmail, IdRegion) WHERE Activo = 1 a nivel de base de datos — el segundo INSERT lanza SQLException de violación de constraint.
6. Mailbot.Api captura específicamente esa excepción (no cualquier SQLException), la trata igual que el paso 4 (no publica, responde 200 OK) y registra un log Information — **no Error** — con el identificadorCorreoGmail duplicado. No se envía alerta Brevo: es comportamiento esperado, no una falla.

Por qué 200 y no 409: Pub/Sub no distingue códigos de error — cualquier respuesta fuera del rango 2xx dispara reintento. Responder 200 ante un duplicado es lo que hace el mecanismo idempotente desde la perspectiva de Pub/Sub.

Detalle — Entidades insertadas al persistir el correo (paso 12)

Tabla	Cardinalidad por correo	Contenido
-------	-------------------------	-----------



CorreoRecibido	1	Metadatos del correo: IdRegion, Asunto, CorreoEmisor, FechaRecepcion. Mantiene la FK IdCorreoRecibidoContenido (NOT NULL) hacia el contenido.
CorreoRecibidoContenido	1	Cuerpo del mensaje (texto/HTML) — separado de CorreoRecibido para no inflar la fila principal. Confirmado 1:1 contra BD real (3706 filas vs 3705 CorreoRecibido) — se crea primero y CorreoRecibido la referencia por FK.
CorreoRecibidoCliente	1	Clasificación (IdCatClasificacionCorreoRecibido) y cliente identificado (IdCliente, nullable — Caso 2). El esquema permite 1:N pero en R16 cada correo tiene exactamente una clasificación.
CorreoRecibidoEstatus	1 (en la inserción inicial) — la tabla es 1:N, no 1:1	Estado operativo: Leido/FechaLeido, Procesado/FechaProcesado, por IdCatClasificacionCorreoRecibido y IdUsuario. Confirmado contra BD real: 8325 filas para 3705 correos (hasta 11 filas para un mismo correo) — es una tabla de auditoría insert-only, no un snapshot. El código legacy (GeneradorProcesoMailBotTransaccionB0.cs, MarcadorLeidoMailBotB0.cs) inserta una fila nueva (IdCorreoRecibidoEstatus = Guid.Empty → INSERT, nunca UPDATE) en dos casos: al clasificar/reclasificar un correo (una fila por clasificación), y cada vez que un usuario distinto marca el correo como leído. Para



		FU-008 (Flujo 1) basta insertar una fila inicial por correo; si se implementa "marcar leído" o reclasificación (Flujo 3), debe insertarse una fila nueva , no actualizar la existente — así se mantiene compatible con el resto del sistema que ya lee esta tabla.
ArchivoCorreoRecibido	0..N	Una fila por adjunto — relación N:N hacia Archivo (MinIO).
Archivo	0..N	Un registro por archivo físico subido a MinIO — tabla compartida del ecosistema, no exclusiva de Mailbot.

Orden de inserción (misma transacción): CorreoRecibidoContenido → CorreoRecibido (con la FK ya resuelta) → CorreoRecibidoCliente → CorreoRecibidoEstatus (1 fila inicial) → Archivo (uno por adjunto) → ArchivoCorreoRecibido (uno por adjunto). Si `clasificacion == 'cobro'`, después de CorreoRecibidoCliente se inserta `fccFolioPagoCliente` (paso 14) — **fuera** de esta transacción, ya que es responsabilidad de `GenerarPendienteUseCase`, no de `ICorreoRepository.GuardarCorreoAsync`.

2. Flujo — Consulta del Buzón de Cobros

1. El Gestor de Cobranza accede a la pantalla de Buzón de Cobros en ProquifaDotNet.
2. El Frontend llama a `POST /BuzonCobros` con los parámetros de paginación y filtros.
3. `BuzonCobrosController` extrae `IdUsuarioCobrador` e `IdRegion` del token JWT.
4. `BuzonCobrosB0` ejecuta la query filtrada por cobrador, región y `crc.Activo = 1`.
5. Retorna lista paginada de correos clasificados como cobro asignados al gestor autenticado.
6. Correos de MEX no aparecen en contexto PER y viceversa (filtro `IdRegion`).

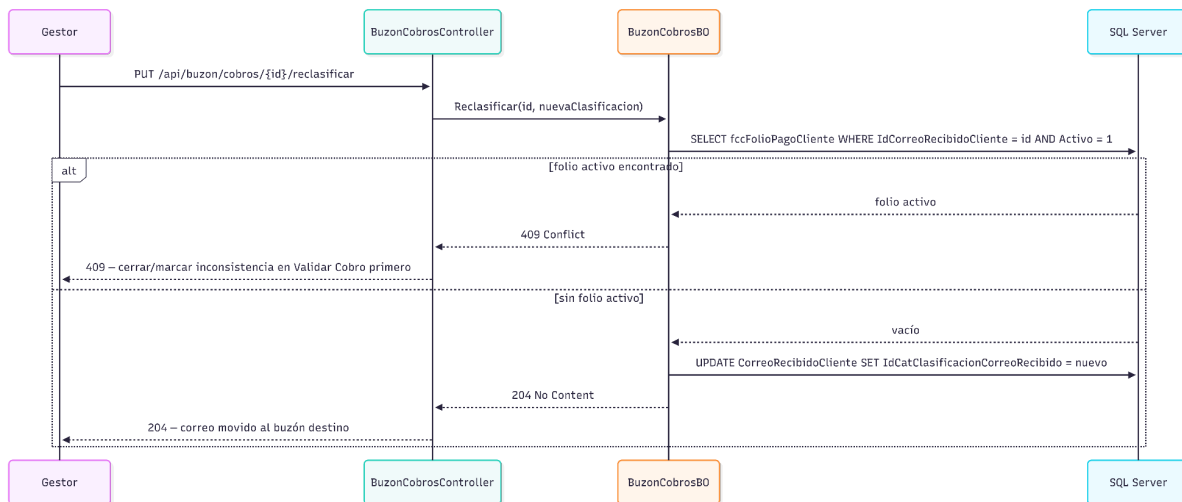
3. Flujo — Reclasificación manual

1. El Gestor identifica un correo mal clasificado en el Buzón de Cobros.
2. El Frontend llama a `PUT /api/buzon/cobros/{id}/reclasificar` con el buzón destino.
3. `BuzonCobrosB0.Reclasificar` verifica si existe `fccFolioPagoCliente.Activo = 1` para ese `IdCorreoRecibidoCliente`.
 - **Si existe** → rechaza con 409 Conflict: *"El correo tiene un pendiente activo en Validar Cobro. Cierre o marque como inconsistencia desde Validar Cobro antes de reclasificar."* No se modifica la clasificación (decisión D1, Opción B).
 - **Si no existe** (folio inexistente o ya cerrado) → continúa al paso 4.



1. BuzonCobrosB0.Reclasificar hace UPDATE en CorreoRecibidoCliente.IdCatClasificacionCorreoRecibido al nuevo valor, e **inserta una fila nueva en CorreoRecibidoEstatus** con la clasificación destino — esta tabla es insert-only, nunca se actualiza una fila existente (mismo patrón que usa GeneradorProcesoMailBotTransaccionB0.cs al reclasificar).
2. El correo desaparece del Buzón de Cobros y aparece en el buzón destino (Pedidos, Cotizaciones u Otros).
3. No existe la opción "marcar como no-cobro" — siempre se mueve a otro buzón existente.

Por qué el bloqueo no genera fricción en la práctica: *un folio que ya fue trabajado y cerrado en Validar Cobro (vinculado a proforma/factura o marcado como inconsistencia) deja Activo = 0 y el correo ya desapareció del Buzón (Regla 6/7) — no hay forma de llegar al paso 2 con ese correo. El único escenario real donde el paso 3 bloquea es mientras el folio sigue sin trabajarse, que es exactamente cuando se quiere forzar el paso por Validar Cobro primero.*



None

sequenceDiagram

participant G as Gestor

participant API as BuzonCobrosController

participant BO as BuzonCobrosB0

participant BD as SQL Server

G->>API: PUT /api/buzon/cobros/{id}/reclasificar

API->>BO: Reclasificar(id, nuevaClasificacion)

BO->>BD: SELECT fccFolioPagoCliente WHERE IdCorreoRecibidoCliente = id AND Activo = 1

alt

folio activo encontrado

BD-->>BO: folio activo

```

BO-->>API: 409 Conflict
API-->>G: 409 - cerrar/marcar inconsistencia en Validar Cobro primero
else sin folio activo
  BD-->>B0: vacío
  BO-->>BD: UPDATE CorreoRecibidoCliente SET IdCatClasificacionCorreoRecibido
= nuevo
  BO-->>API: 204 No Content
  API-->>G: 204 - correo movido al buzón destino
end

```

4. Flujo — Cierre y eliminación del pendiente

PUT /api/cobros/folio/{id}/cerrar lo llama el **frontend de Validar Cobro** directamente vía HTTP, con el mismo token JWT del Gestor ya autenticado en esa pantalla — no es una llamada interna BO-a-BO ni un evento asíncrono entre módulos. Aunque Validar Cobro y Buzón de Cobros viven en el mismo backend monolítico (ProquifaDotNet), cada pantalla llama a su propio endpoint REST, igual que el resto de los módulos existentes del sistema. FolioPagoClienteB0.Cierre no se invoca nunca directamente desde el código de Validar Cobro — solo a través de este endpoint HTTP.

Validar Cobro no es un módulo único — es un wizard de 3 pasos (RE-023 Listado, RE-024/025 Paso 1 Captura, RE-026/027 Paso 2 Asociación, RE-028/029 Paso 3 Facturación). El módulo que invoca este endpoint es específicamente **Finanzas**, en el momento en que el usuario confirma la Asociación (fin del Paso 2 — botón "Continuar"), sin importar si el resultado fue pago exacto, sobrepago o pago con tolerancia. Esos sub-requisitos (RE-023 a RE-030) están fuera del alcance de FU-008 y todavía no tienen tareas propias definidas.

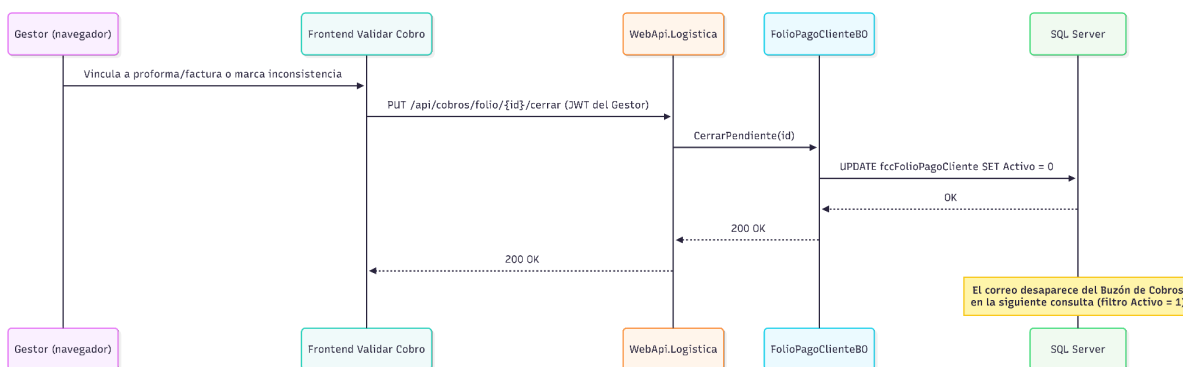
Cierre (vinculación a proforma/factura en Validar Cobro):

1. El Gestor vincula el cobro a una proforma o factura en el módulo Validar Cobro (FU-023).
2. El frontend de Validar Cobro llama PUT /api/cobros/folio/{id}/cerrar.
3. FolioPagoClienteB0.Cierre ejecuta UPDATE fccFolioPagoCliente SET Activo = 0.
4. El pendiente desaparece automáticamente del Buzón de Cobros (query filtra Activo = 1).

Eliminación (inconsistencia en Validar Cobro):

1. El Gestor marca el cobro como inconsistencia en Validar Cobro.
2. El frontend de Validar Cobro llama PUT /api/cobros/folio/{id}/cerrar — mismo endpoint, sin distinguir el motivo (decisión D2).

3. FolioPagoClienteB0.Cierre ejecuta UPDATE fccFolioPagoCliente SET Activo = 0.
4. El pendiente desaparece del Buzón de Cobros. El tratamiento de la inconsistencia pertenece a FU-023.



None

sequenceDiagram

```

participant G as Gestor (navegador)
participant VC as Frontend Validar Cobro
participant API as WebApi.Logistica
participant B0 as FolioPagoClienteB0
participant BD as SQL Server
  
```

```

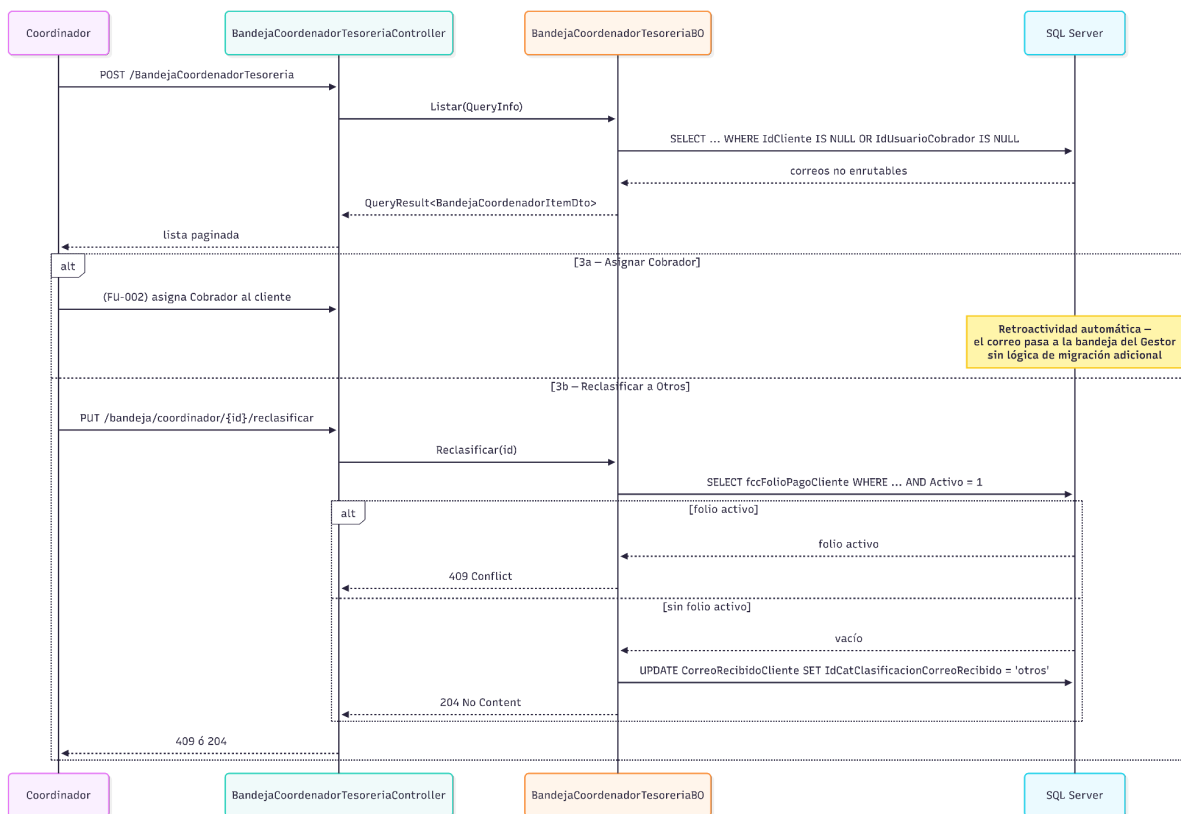
G->>VC: Vincula a proforma/factura o marca inconsistencia
VC->>API: PUT /api/cobros/folio/{id}/cerrar (JWT del Gestor)
API->>B0: CerrarPendiente(id)
B0->>BD: UPDATE fccFolioPagoCliente SET Activo = 0
BD-->>B0: OK
B0-->>API: 200 OK
API-->>VC: 200 OK
  
```

Note over BD: El correo desaparece del Buzón de Cobros en la siguiente consulta (filtro Activo = 1)

5. Flujo — Bandeja del Coordinador de Tesorería (OBS-021)

1. Un correo clasificado como cobro no puede enrutarse a ningún Gestor por uno de dos motivos:
 - a. **Caso 1:** cliente existe pero sin Cobrador asignado (ClienteCartera.IdUsuarioCobrador IS NULL).
 - b. **Caso 2:** remitente no dado de alta como contacto de ningún cliente (CorreoRecibidoCliente.IdCliente IS NULL).
2. El correo queda visible en la Bandeja del Coordinador de Tesorería (POST /BandejaCoordenadorTesoreria).
3. El Coordinador tiene dos acciones disponibles:

- a. **3a. Asignar Cobrador** al cliente (FU-002). La query del Gestor filtra por IdUsuarioCobrador: el correo pasa automáticamente a la bandeja del Gestor sin lógica adicional de migración (retroactividad por diseño).
- b. **3b. Reclasificar a "Otros"** (decisión D3, Opción B) — PUT /bandeja/coordinador/{id}/reclasificar. Fallback para cuando el Agente IA clasificó incorrectamente el correo como Cobro; no es la ruta esperada, pero debe estar disponible. Aplica la misma validación de folio activo que D1: si fccFolioPagoCliente.Activo = 1, responde 409 Conflict (el correo no enrutable también genera folio automáticamente — Regla 4). El Coordinador **no puede** reclasificar a Cobros, Pedidos ni Cotizaciones desde esta bandeja — únicamente a Otros.



None

sequenceDiagram

```

participant C as Coordinador
participant API as BandejaCoordinadorTesoreriaController
participant BO as BandejaCoordinadorTesoreriaBO
participant BD as SQL Server

```

C->>API: POST /BandejaCoordinadorTesoreria

API->>BO: Listar(QueryInfo)

```

BO-->BD: SELECT ... WHERE IdCliente IS NULL OR IdUsuarioCobrador IS NULL
BD-->BO: correos no enrutables
BO-->API: QueryResult<BandejaCoordinadorItemDto>
API-->C: lista paginada

alt 3a - Asignar Cobrador
  C-->API: (FU-002) asigna Cobrador al cliente
  Note over BD: Retroactividad automática -->el correo pasa a la bandeja del
  Gestor-->sin lógica de migración adicional
  else 3b - Reclasificar a Otros
    C-->API: PUT /bandeja/coordinador/{id}/reclasificar
    API-->BO: Reclasificar(id)
    BO-->BD: SELECT fccFolioPagoCliente WHERE ... AND Activo = 1
    alt folio activo
      BD-->BO: folio activo
      BO-->API: 409 Conflict
    else sin folio activo
      BD-->BO: vacío
      BO-->BD: UPDATE CorreoRecibidoCliente SET IdCatClasificacionCorreoRecibido =
      'otros'
      BO-->API: 204 No Content
    end
  end
  API-->C: 409 ó 204
end

```

6. Criterios de aceptación del requisito

CA	Descripción	Estado	Justificación
CA-01	catClasificacionCorreoRecibido tiene registro con Clave = 'cobro' y AnalistaDeCuentasPorCobrar = 1	Pendiente	Tarea T1 — INSERT nuevo registro (Opción A, no se renombra pago)
CA-02	RegionConfiguracionMailBot tiene columnas PubSubTopic, WatchHistoryId, WatchExpiration	Pendiente	Tarea T1 (script BD)
CA-03	MailbotEventoCorreo y MailbotClasificacionLog creadas con índices correctos	Pendiente	Tareas T2/T3
CA-04	POST /BuzonCobros retorna listado paginado filtrado por cobrador e IdRegion del token	Pendiente	Tarea T9
CA-05	Correos MEX no aparecen en buzón de cobrador PER y viceversa	Pendiente	Cubierto en BuzonCobrosB0 — filtro IdRegion

CA-06	PUT /api/buzon/cobros/{id}/reclasificar mueve el correo al buzón destino	Pendiente	Tarea T10
CA-06b	PUT /api/buzon/cobros/{id}/reclasificar responde 409 Conflict y no modifica la clasificación si existe fccFolioPagoCliente.Activo = 1 para ese correo	Pendiente	Tarea T10 — decisión D1 (Opción B)
CA-07	PUT /api/cobros/folio/{id}/cerrar cierra el pendiente (fccFolioPagoCliente.Activo = 0)	Pendiente	Tarea T11
CA-08	Buzones de Cotizaciones y Pedidos funcionan con el mismo patrón	Pendiente	Tareas T25-T32
CA-09	POST /webhook/gmail/{region} responde 200 OK y publica en Channel interno	Pendiente	Tarea T20
CA-10	PubSubTokenValidationMiddleware rechaza tokens inválidos con 401	Pendiente	Tarea T20
CA-11	CorreoWorkerMex y CorreoWorkerPer consumen el Channel de forma independiente	Pendiente	Tarea T21
CA-12	GmailWatchRenovacionWorker renueva el watch antes de expirar (cada 6 días)	Pendiente	Tarea T21
CA-13	Agente IA clasifica correctamente por asunto, cuerpo y adjuntos (cotizacion / pedido / cobro / otros)	Pendiente	Tarea T15 + prompts (pendiente de validación de negocio)
CA-14	Al clasificar como cobro se genera fccFolioPagoCliente con campos *MailBot pre-cargados	Pendiente	Tarea T19 (GenerarPendienteUseCase)
CA-15	Los adjuntos se guardan en MinIO antes de persistir en BD	Pendiente	Tarea T17
CA-16	Errores envían alerta vía Brevo al equipo de operaciones	Pendiente	Tarea T18



CA-17	MailbotEventoCorreo registra cada evento con Procesado, Intentos y Error	Pendiente	Tarea T20
CA-18	MailbotClasificacionLog registra cada clasificación con confianza, versión del modelo y tokens	Pendiente	Tarea T19
CA-19	Backoff exponencial: 5s, 10s, 20s antes de NACK → DLQ	Pendiente	Tarea T21
CA-20	MailbotWorker.sln compila en .NET 10 sin errores	Pendiente	Tarea T13
CA-21	PUT /bandeja/coordinador/{id}/reclasificar mueve el correo al buzón "Otros"; rechaza cualquier otro destino	Pendiente	Tarea T37 — decisión D3 (Opción B)
CA-22	PUT /bandeja/coordinador/{id}/reclasificar responde 409 Conflict y no modifica la clasificación si existe fccFolioPagoCliente.Activo = 1 para ese correo	Pendiente	Tarea T37 — misma regla que CA-06b (RT-11)
CA-23	GET /admin/mailbot/eventos?estado=no-procesado retorna lista de correos con Procesado = 0 y Error no nulo; POST /admin/mailbot/eventos/{id}/retry resetea Intentos = 0 y republica al Channel; POST /admin/mailbot/eventos/{id}/descartar registra MotivoRechazo = OtroMotivo y Procesado = 1	Pendiente	Tarea T21 (endpoints admin)
CA-24	MailbotClasificacionLog tiene fila para cada intento de clasificacion del Agente IA — incluyendo fallos con ClasificacionIA = NULL y TipoFallo poblado	Pendiente	Tareas T15 + T21 (RT-21)

7. Reglas técnicas aplicadas



Regla	Descripción
RT-01	El switch de clasificación en <code>GeneradorProcesoMailBotB0.cs</code> DEBE evaluar la columna <code>Clave</code> de <code>catClasificacionCorreoRecibido</code> , no el nombre <code>display</code> . Bug L3 del legacy que se corrige.
RT-02	La idempotencia se garantiza mediante <code>MailbotEventoCorreo.IdentificadorCorreoGmail</code> (índice <code>UNIQUE</code>). Si Pub/Sub reintenta el mismo push, el <code>INSERT</code> falla silenciosamente y no se procesa dos veces.
RT-03	El watch de Gmail expira a los 7 días. <code>GmailWatchRenovacionWorker</code> lo renueva cada 6 días y actualiza <code>RegionConfiguracionMailBot.WatchExpiration</code> .
RT-04	<code>Mailbot.Api</code> DEBE responder 200 OK inmediatamente tras encolar en el Channel. Si responde cualquier error 5xx, Pub/Sub reintentará el push hasta agotar los 5 intentos configurados.
RT-05	El backoff ante errores del Worker es exponencial: 5s → 10s → 20s. Agotados los reintentos → NACK → DLQ <code>mailbot-dlq</code> + alerta Brevo.
RT-06	Los prompts del Agente IA son archivos <code>.txt</code> en <code>Mailbot.Infrastructure/Prompts/</code> . Son el único mecanismo de ajuste de precisión del clasificador. No se exponen en la interfaz.
RT-07	La visibilidad del Buzón de Cobros se filtra exclusivamente por <code>ClienteCartera.IdUsuarioCobrador</code> extraído del token JWT. El Gestor solo ve correos de su cartera asignada.
RT-08	La segregación regional se aplica mediante <code>CorreoRecibido.IdRegion</code> . Correos MEX (<code>IdRegion = 1</code>) no aparecen en consultas de contexto PER (<code>IdRegion = 2</code>) y viceversa.
RT-09	Los campos <code>SubtotalMailBot</code> , <code>IvaMailBot</code> , <code>TotalMailBot</code> , <code>MxnMailBot</code> , <code>UsdMailBot</code> de <code>fccFolioPagoCliente</code> DEBEN pre-cargarse desde la extracción del Agente IA al crear el folio. No se dejan en NULL.
RT-10	El Gestor no puede eliminar correos directamente. La única acción de retiro es la reclasificación a otro buzón. La eliminación la realiza el rol ESAC desde el buzón Otros.
RT-11	La reclasificación manual (<code>PUT /api/buzon/cobros/{id}/reclasificar</code>) DEBE verificar <code>fccFolioPagoCliente.Activo</code> antes de actualizar la clasificación. Si <code>Activo = 1</code> , responde 409 Conflict sin modificar nada (decisión D1, Opción B — bloqueo, no anulación automática).
RT-12	El Coordinador de Tesorería y el Gerente de Tesorería solo pueden reclasificar correos de la Bandeja del Coordinador al destino "Otros". Ninguno de los dos tiene acceso a reclasificar hacia Cobros, Pedidos ni Cotizaciones (decisión D3, Opción

	B). P16 resuelta 2026-06-19 (decisión de Carlos): por simetría con OBS-003 (el cliente ya amplió a Gerente de Tesorería el permiso de asignar Cobrador en FU-002, mismo criterio de autorización), POST /BandejaCoordinadorTesoreria y PUT /bandeja/coordinador/{id}/reclasificar validan ambos roles. El campo exacto en Usuario para "Gerente de Tesorería" todavía no existe (GerenteDeTesoreria hoy representa a Coordinador) — agregarlo es un nuevo bit siguiendo el mismo patrón que las 39 columnas de rol ya existentes en la tabla (confirmado contra BD real, instancia DEV), sin riesgo ni cambio de diseño. El nombre definitivo de esa columna sigue siendo GAP-04/OBS-003 de FU-002 (Juan David/Ryndem) — no bloquea T36/T37, que añaden el OR de autorización en cuanto el campo exista.
RT-13	PUT /bandeja/coordinador/{id}/reclasificar aplica la misma validación de folio activo que RT-11 (fccFolioPagoCliente.Activo = 1 → 409 Conflict) — un correo no enrutable también genera folio automáticamente al clasificarse como cobro (Regla 4).
RT-14	GeminiClasificadorAgente DEBE usar el modo de salida estructurada de Vertex AI (GenerationConfig.ResponseMimeType = "application/json" + ResponseSchema), no parsear texto libre. Un fallo de parseo se trata como error de clasificación (RT-05) — no se asume un valor por defecto.
RT-15	El corte de ConsumidorMailBot (legacy) es único y simultáneo para los 4 tipos de clasificación — NUNCA escalonado por buzón. Se apaga en el mismo despliegue donde MailbotWorker.sln entra en producción (decisión D5, Tarea T38).
RT-16	En Desarrollo, los eventos de Gmail se reciben vía subscription Pull (GmailPubSubPullListener) en vez de Push — NUNCA se expone Mailbot.Api públicamente solo para dev. El permiso IAM de Gmail hacia Pub/Sub SIEMPRE se otorga a gmail-api-push@system.gserviceaccount.com (cuenta de Google), nunca a una service account del proyecto (Tarea T39).
RT-17	MailbotWorker.sln NO aplica etiquetas de Gmail en ningún paso del flujo. Gmail es exclusivamente el canal de entrada — la notificación push llega, el sistema la procesa y el estado queda en la BD (MailbotEventoCorreo, CorreoRecibidoCliente). El control de correos procesados, la idempotencia y la trazabilidad son responsabilidad del sistema, no de Gmail. El legacy usaba etiquetas porque operaba por polling IMAP y necesitaba marcar lo ya leído; con Pub/Sub push esa necesidad desaparece.
RT-18	catClasificacionCorreoRecibido WHERE Clave = 'otros' DEBE tener AnalistaDeCuentasPorCobrar = 1 para que el Coordinador/Gerente de Tesorería vea los correos no enrutables en su bandeja. Requiere UPDATE catClasificacionCorreoRecibido SET AnalistaDeCuentasPorCobrar = 1 WHERE Clave = 'otros' (parte de Tarea T1). Efecto colateral conocido y aceptado: correos otros de Cotizaciones y Pedidos también aparecerán en la bandeja de Tesorería — decisión de Carlos 2026-06-25.

RT-19	MailbotEventoCorreo tiene columnas MotivoRechazo NVARCHAR(50) NULL y NotaRechazo NVARCHAR(500) NULL. Valores validos de MotivoRechazo: Spam (lista negra, automatico), ErrorTecnico (DLQ tras agotar reintentos, automatico), OtroMotivo (descarte manual via endpoint admin, acompaña NotaRechazo), Expirado (auto-descarte a 30 dias sin accion). Un correo sin MotivoRechazo esta en proceso o fue procesado exitosamente.
RT-20	Correos en estado NO PROCESADO (Procesado = 0 tras agotar reintentos) tienen retencion de 30 dias. Al dia 15 sin accion, Brevo envia recordatorio al equipo de operaciones. Al dia 30, el sistema asigna MotivoRechazo = 'Expirado' y Procesado = 1 automaticamente. Las acciones disponibles (retry / descarte con motivo) se ejecutan via endpoint admin sin UI en el front.
RT-21	MailbotClasificacionLog registra TODOS los intentos de clasificacion del Agente IA — exitosos y fallidos. Columnas adicionales: ErrorIA NVARCHAR(500) NULL (mensaje del error si el LLM fallo) y TipoFallo NVARCHAR(50) NULL (valores validos: timeout / parse-error / rate-limit / error-generico). El INSERT ocurre siempre: camino exitoso → ErrorIA = NULL, TipoFallo = NULL; camino fallido → ClasificacionIA = NULL, ErrorIA y TipoFallo se populan (Flujo 1, paso 17). Permite responder "?por que este correo no aparecio en ningun buzón?" consultando BD, sin necesidad de revisar logs de aplicacion.

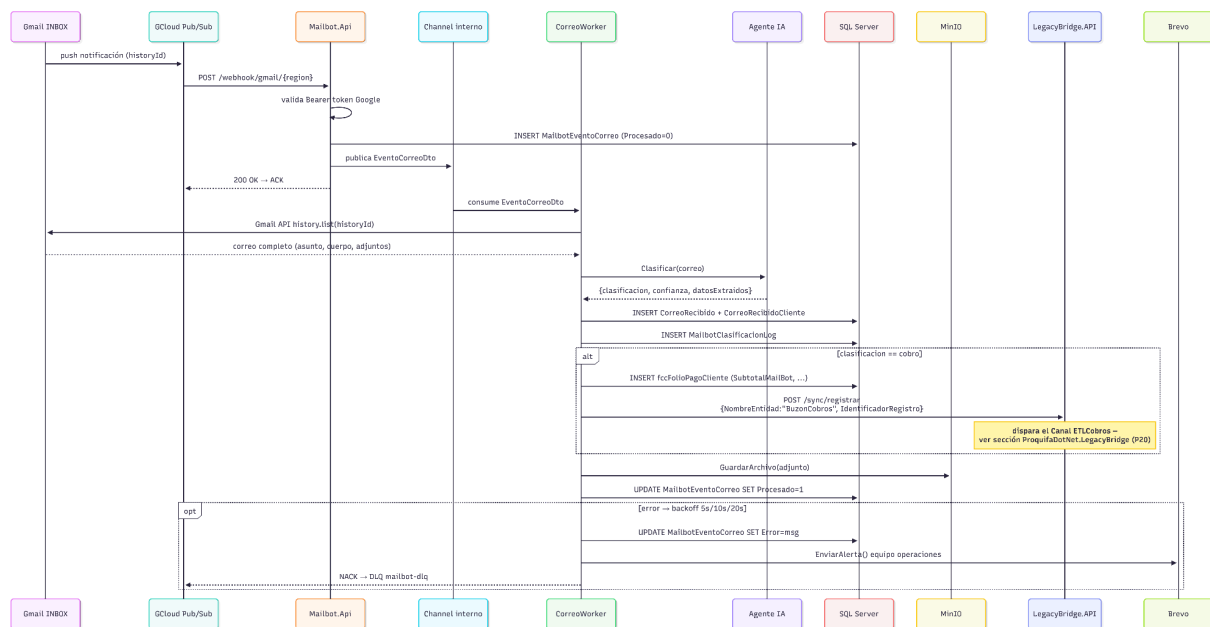


Diseño de componentes

1. Diagramas

Además de los diagramas Mermaid de esta sección, **TPSC-RE-FU-008-Diagramas.drawio** contiene dos vistas complementarias en formato draw.io: la página **"1. Dependencias entre soluciones"** (mapa de tareas y pendientes entre ProquifaDotNet, MailbotWorker.sln y ProquifaDotNet.LegacyBridge, útil para planeación) y la página **"2. C4 - Contenedores (post-implementación)"** (diagrama C4 de nivel 2 con el estado final de las tres soluciones, sus bases de datos, sistemas externos y actores). La presentación ejecutiva de este diseño está en **TPSC-RE-FU-008-DIS-SOL-Presentacion**.

Flujo end-to-end (Gmail Push → Buzón del Gestor)



None

sequenceDiagram

```

participant G as Gmail INBOX
participant PS as GCloud Pub/Sub
participant API as Mailbot.Api
participant CH as Channel interno
participant W as CorreoWorker
participant AI as Agente IA
participant BD as SQL Server
participant MN as MinIO
participant LB as LegacyBridge.API

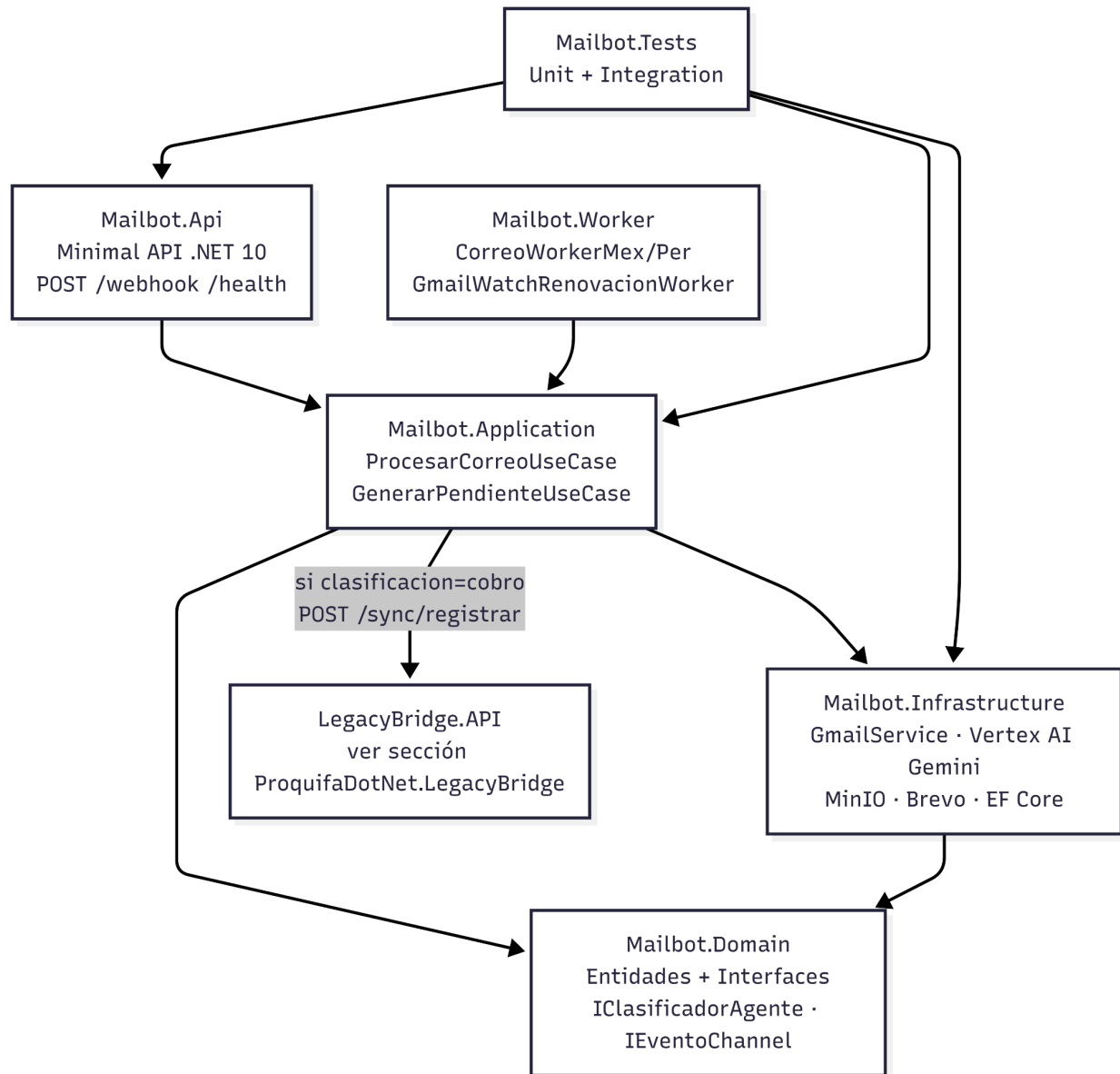
```

```
participant BR as Brevo

G->>PS: push notificación (historyId)
PS->>API: POST /webhook/gmail/{region}
API->>API: valida Bearer token Google
API->>BD: INSERT MailbotEventoCorreo (Procesado=0)
API->>CH: publica EventoCorreoDto
API-->>PS: 200 OK → ACK
CH->>W: consume EventoCorreoDto
W->>G: Gmail API history.list(historyId)
G-->>W: correo completo (asunto, cuerpo, adjuntos)
W->>AI: Clasificar(correo)
AI-->>W: {clasificacion, confianza, datosExtraidos}
W->>BD: INSERT CorreoRecibido + CorreoRecibidoCliente
W->>BD: INSERT MailbotClasificacionLog
alt clasificacion == cobro
    W->>BD: INSERT fccFolioPagoCliente (SubtotalMailBot, ...)
    W->>LB: POST /sync/registrar<br/>{NombreEntidad:"BuzonCobros",
IdentificadorRegistro}
    Note over LB: dispara el Canal ETLCobros -<br/>ver sección
ProquifaDotNet.LegacyBridge (P20)
end
W->>MN: GuardarArchivo(adjunto)
W->>BD: UPDATE MailbotEventoCorreo SET Procesado=1
opt error → backoff 5s/10s/20s
    W->>BD: UPDATE MailbotEventoCorreo SET Error=msg
    W->>BR: EnviarAlerta() equipo operaciones
    W-->>PS: NACK → DLQ mailbot-dlq
end
```



Arquitectura de la solución MailbotWorker.sln



None

graph TD

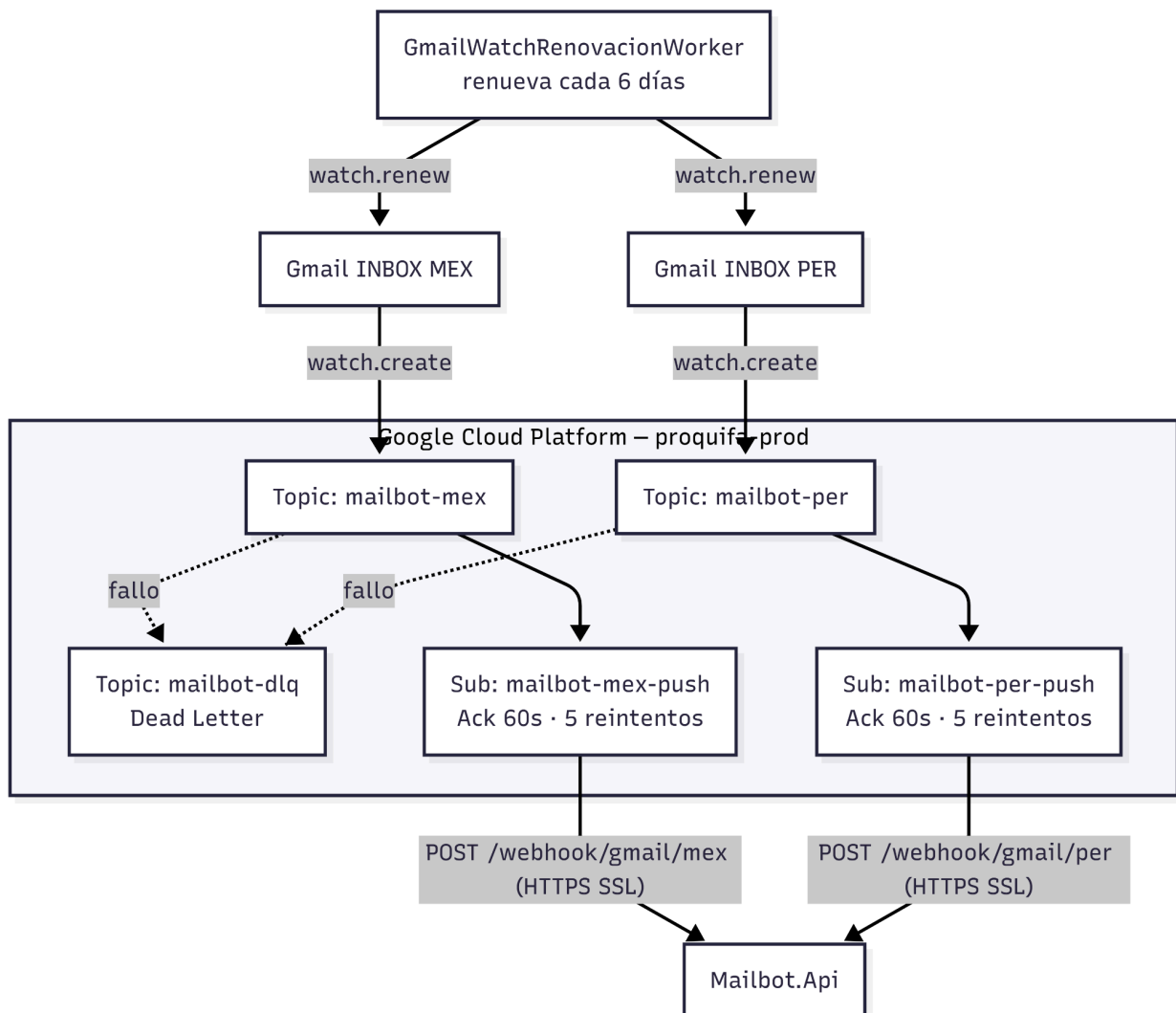
```

API[Mailbot.Api<br/>Minimal API .NET 10<br/>POST /webhook /health] --> APP
WRK[Mailbot.Worker<br/>CorreoWorkerMex/Per<br/>GmailWatchRenovacionWorker] --> APP
APP[Mailbot.Application<br/>ProcesarCorreoUseCase<br/>GenerarPendienteUseCase] --> DOM
APP --> INF
INF[Mailbot.Infrastructure<br/>GmailService · Vertex AI Gemini<br/>MinIO · Brevo · EF Core] --> DOM
DOM[Mailbot.Domain<br/>Entidades + Interfaces<br/>IClasificadorAgente · IEventoChannel]
TST[Mailbot.Tests<br/>Unit + Integration] --> API
  
```

```
TST --> APP
TST --> INF
APP -->|"si clasificacion=cobro">POST /sync/registrar"| LBAPI[LegacyBridge.API<br/>ver
sección ProquifaDotNet.LegacyBridge]
```

Despliegue: convención del arquetipo microservice-clean-architecture-template — Mailbot.Worker se instala como **Windows Service** (sc.exe create) en todos los ambientes; Mailbot.Api se publica como **sitio IIS**, pero **solo aplica a Staging/Prod** (mismo IIS donde ya corre ProquifaDotNet). En Desarrollo Mailbot.Api no se levanta — GmailPubSubPullListener, dentro de Mailbot.Worker, consume la subscription Pull directamente (ver "Estrategia por ambiente" más abajo).

Infraestructura Google Cloud Pub/Sub



None

flowchart TB

```

subgraph GCP["Google Cloud Platform - proquifa-prod"]
    TM[Topic: mailbot-mex]
    TP[Topic: mailbot-per]
    DLQ[Topic: mailbot-dlq<br/>Dead Letter]
    SM[Sub: mailbot-mex-push<br/>Ack 60s · 5 reintentos]
    SP[Sub: mailbot-per-push<br/>Ack 60s · 5 reintentos]
    TM --> SM
    TP --> SP
    TM -.->|fallo| DLQ
    TP -.->|fallo| DLQ
end
GM[Gmail INBOX MEX] -->|watch.create| TM
GP[Gmail INBOX PER] -->|watch.create| TP
SM -->|"POST /webhook/gmail/mex (HTTPS SSL)"| MAPI[Mailbot.Api]
SP -->|"POST /webhook/gmail/per (HTTPS SSL)"| MAPI
RW[GmailWatchRenovacionWorker<br/>renueva cada 6 días] -->|watch.renew| GM
RW -->|watch.renew| GP

```

Permiso IAM correcto: *Gmail publica al topic usando la cuenta administrada por Google gmail-api-push@system.gserviceaccount.com — no una service account propia del proyecto. Hay que otorgar roles/pubsub.publisher en mailbot-mex/mailbot-per/mailbot-dlq a esa cuenta exacta; si se otorga a una cuenta del proyecto, watch.create responde 200 OK pero nunca llega ninguna notificación (falla silenciosa, causa más común de que esta integración no funcione).*

Estrategia por ambiente — Pull (Desarrollo) vs Push (Staging/Prod)

*Pub/Sub Push requiere un endpoint HTTPS público — no disponible ni en laptop local ni en el servidor DEV sin involucrar a IT/Redes. En Desarrollo se usa una **subscription Pull** en su lugar; Staging/Prod siguen usando Push tal como en el diagrama anterior.*

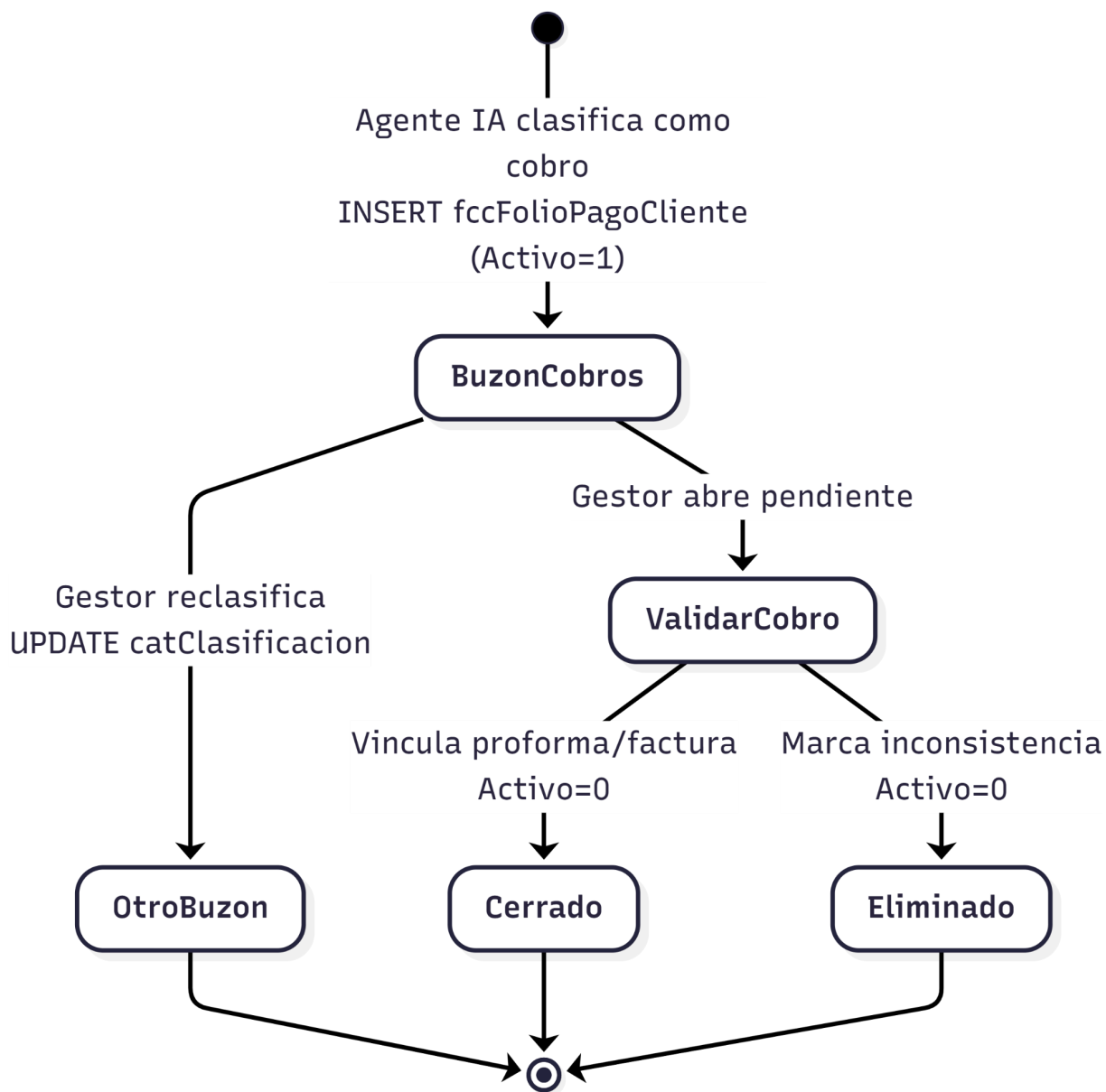
	Desarrollo	Staging / Producción
Tipo de subscription	Pull (mailbot-mex-pull, mailbot-per-pull)	Push (mailbot-mex-push, mailbot-per-push)
Quién consume	GmailPubSubPullListener (SubscriberClient, polling activo dentro de Mailbot.Worker)	Mailbot.Api (POST /webhook/gmail/{region})
Requiere endpoint público	No	Sí — PubSubTokenValidationMiddleware valida el Bearer token de Google



Provisiona	Carlos (proyecto GCP propio de dev)	DevOps
Destino tras recibir el mensaje	Mismo interno IEventoChannel	Mismo interno IEventoChannel

El resto del pipeline (Flujo 1, pasos 9 en adelante) es idéntico en ambos ambientes — solo cambia el adaptador de entrada. Tarea: **T39** (Plan de implementación, Ola 6).

Ciclo de vida del pendiente en el Buzón



None

stateDiagram-v2

[*] --> BuzonCobros: Agente IA clasifica como cobro
INSERT fccFolioPagoCliente (Activo=1)

BuzonCobros --> OtroBuzon: Gestor reclasifica
UPDATE catClasificacion

BuzonCobros --> ValidarCobro: Gestor abre pendiente

ValidarCobro --> Cerrado: Vincula proforma/factura
Activo=0

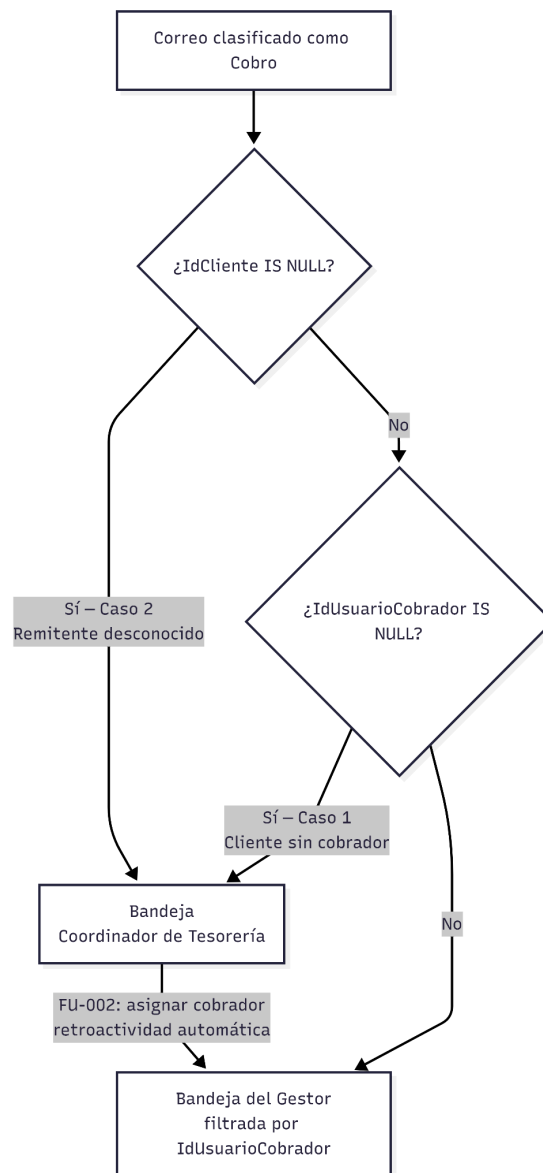
ValidarCobro --> Eliminado: Marca inconsistencia
Activo=0

OtroBuzon --> [*]

Cerrado --> [*]

Eliminado --> [*]

Bandeja del Coordinador de Tesorería (OBS-021)



None

flowchart TD

```

CR[Correo clasificado como Cobro] --> Q1{¿IdCliente IS NULL?}
Q1 -->|Sí - Caso 2<br/>Remitente desconocido| COORD[Bandeja<br/>Coordinador de Tesorería]
Q1 -->|No| Q2{¿IdUsuarioCobrador IS NULL?}
Q2 -->|Sí - Caso 1<br/>Cliente sin cobrador| COORD
Q2 -->|No| GESTOR[Bandeja del Gestor<br/>filtrada por IdUsuarioCobrador]
COORD -->|FU-002: asignar cobrador<br/>retroactividad automática| GESTOR

```

2. Contratos de endpoints

Tipos base reutilizados de Core.CrudTools.Optimization (ya usados por todos los Buzones existentes) — no se definen de nuevo: QueryInfo (PageNumber, PageSize, Filters: List<FilterTuple>, SortField, SortDirection) y QueryResult<T> (Items: List<T>, TotalCount: int, PageNumber: int, PageSize: int).

POST /BuzonCobros

Propiedad	Descripción
Request	QueryInfo — filtros típicos: Leido, rango de FechaRecepcion, texto libre sobre Asunto/CorreoEmisor
Contexto implícito	IdUsuarioCobrador e IdRegion se extraen del token JWT — no van en el body
Response 200	QueryResult<BuzonCobrosDetalleDto>

BuzonCobrosDetalleDto:

Campo	Tipo	Nulo	Origen
IdCorreoRecibido	Guid	No	CorreoRecibido.IdCorreoRecibido
Asunto	string	No	CorreoRecibido.Asunto
CorreoEmisor	string	No	CorreoRecibido.CorreoEmisor
FechaRecepcion	DateTime	No	CorreoRecibido.FechaRecepcion
Cliente	string	No	Cliente.Nombre
Region	string	No	Region.ClaveISO (MEX / PER)
FolioCobro	string	Sí	fccFolioPagoCliente.Folio
TotalMailBot	decimal	Sí	fccFolioPagoCliente.TotalMailBot
Leido	bool	No	CorreoRecibidoCliente.Leido

Procesado	bool	No	CorreoRecibidoCliente.Procesado
-----------	------	----	---------------------------------

PUT /api/buzon/cobros/{id}/reclasificar

Propiedad	Descripción
Path	id: Guid — IdCorreoRecibidoCliente
Request body	GMReclasificarCorreo — { "IdCatClasificacionCorreoRecibido": "guid" }
Response	204 No Content si se reclasifica; 404 si id no existe; 400 si IdCatClasificacionCorreoRecibido no corresponde a una clasificación válida; 409 Conflict si existe fccFolioPagoCliente.Activo = 1 para ese correo (decisión D1)

PUT /api/cobros/folio/{id}/cerrar

Propiedad	Descripción
Path	id: Guid — IdFCCFolioPagoCliente
Request body	Ninguno — ver decisión D2: un solo endpoint, sin IdProforma/IdFactura, sin campo motivo
Response	200 OK si se cierra; 404 si el folio no existe o ya estaba Activo = 0

POST /BandejaCoordinadorTesoreria

Propiedad	Descripción
Request	QueryInfo — mismo patrón que POST /BuzonCobros
Autorización	Solo rol Coordinador de Tesorería — 403 para cualquier otro rol
Response 200	QueryResult<BandejaCoordinadorItemDto>

BandejaCoordinadorItemDto:

Campo	Tipo	Nulo	Notas
IdCorreoRecibido	Guid	No	
Asunto	string	No	
CorreoEmisor	string	No	
FechaRecepcion	DateTime	No	
IdCorreoRecibidoCliente	Guid	No	
IdCliente	Guid	Sí	NULL en Caso 2
Cliente	string	Sí	NULL en Caso 2

MotivoBandejaCoordinador	string	No	"Caso 1 - Cliente sin Cobrador asignado" O "Caso 2 - Remitente no dado de alta"
Region	string	No	
IdFCCFolioPagoCliente	Guid	Sí	
FolioCobro	string	Sí	

PUT /bandeja/coordinador/{id}/reclasificar

Decisión D3 (Opción B) — fallback ante clasificación incorrecta de la IA; no es la ruta esperada del flujo.

Propiedad	Descripción
Path	id: Guid — IdCorreoRecibidoCliente
Request body	Ninguno — el único destino válido es "Otros", no se recibe IdCatClasificacionCorreoRecibido
Autorización	Solo rol Coordinador de Tesorería — 403 para cualquier otro rol
Response	204 No Content si se reclasifica a Otros; 404 si id no existe; 409 Conflict si existe fccFolioPagoCliente.Activo = 1 para ese correo (misma regla que D1 — RT-11)

3. Interfaces del dominio

```
CSharp
// Mailbot.Domain/Interfaces/IClasificadorAgente.cs
// Implementa: Mailbot.Infrastructure/IA/GeminiClasificadorAgente.cs
public interface IClasificadorAgente
{
    Task<ResultadoClasificacionDto> ClasificarAsync(CorreoCompletoDto correo,
    CancellationToken ct = default);
}

public record CorreoCompletoDto(string Asunto, string CorreoEmisor, string Cuerpo,
    IReadOnlyList<AdjuntoDto> Adjuntos);

public record ResultadoClasificacionDto(
    string Clasificacion, // "cotizacion" | "pedido" | "cobro" | "otros"
    decimal Confianza,
    object DatosExtraidos, // DTO específico por tipo, ej. DatosExtraidosCobroDto
    (Subtotal, Iva, Total, Mxn, Usd)
```



```
string ModeloVersion,
int PromptTokens,
int CompletionTokens);
```

Contrato JSON del Agente IA

Shape exacto esperado entre PromptBuilder y la respuesta del LLM. Proveedor confirmado — P5 resuelta 2026-06-19: Gemini Flash-Lite vía Vertex AI (ver Alcance).

Recomendación técnica: usar el *structured output* de Vertex AI (`GenerationConfig.ResponseMimeType = "application/json" + ResponseSchema`) en vez de pedir JSON por instrucción de texto libre en el prompt. Un LLM en modo texto libre puede devolver prosa adicional o JSON mal formado; `GeminiClasificadorAgente` no debería depender de un parser tolerante a fallos para esto — debe exigir el modo estructurado de Vertex AI. Si una respuesta llega mal formada de todos modos, `RegistrarErrorAsync` debe tratar un fallo de parseo igual que un error de clasificación (RT-05: backoff → DLQ), nunca asumir un valor por defecto.

Respuesta esperada — siempre presente:

```
JSON
{
  "clasificacion": "cobro",
  "confianza": 0.94,
  "datosExtraidos": { },
  "modeloVersion": "gemini-2.5-flash-lite"
}
```

`promptTokens` y `completionTokens` del `ResultadoClasificacionDto` **no** vienen en el JSON del modelo — los expone el SDK de Vertex AI (`Google.Cloud.AIPlatform.V1`) en `GenerateContentResponse.UsageMetadata` (`PromptTokenCount` / `CandidatesTokenCount`), `GeminiClasificadorAgente` los toma de ahí, no del cuerpo JSON parseado.

datosExtraidos por clasificación (DTOs concretos detrás del object del record):

```
CSharp
// clasificacion == "cobro" - prompt extraccion_cobro.txt
public record DatosExtraidosCobroDto(
    decimal? Subtotal, decimal? Iva, decimal? Total,
    bool Mxn, bool Usd,
    string? BancoEmisor, string? CuentaOrigen,
    DateTime? FechaPago, string? ReferenciaBancaria);

// clasificacion == "pedido" - prompt extraccion_pedido.txt
```

```
public record DatosExtraidosPedidoDto(
    IReadOnlyList<string> Productos, string? Cliente);

// clasificacion == "cotizacion" - prompt extraccion_cotizacion.txt
public record DatosExtraidosCotizacionDto(
    IReadOnlyList<string> ProductosSolicitados, string? Urgencia);

// clasificacion == "otros" - sin prompt de extracción, no se invoca extracción adicional
public record DatosExtraidosOtrosDto();
```

Muestras generadas 2026-06-26: los campos de DatosExtraidosPedidoDto/DatosExtraidosCotizacionDto están validados contra correos reales del dataset legacy (ML.NET, 2020). Archivos de muestra anonimizados para redactar los prompts de clasificación y extracción:

Archivo	Tipo	Patrones	Propósito
TPSC-RE-FU-008-Muestra-Correos-Cobro-Anonimizada.md	cobro	8 (incl. 3 reales 2026)	extraccion_cobro.txt + validar DatosExtraidosCobroDto — 2 formatos bancarios reales
TPSC-RE-FU-008-Muestra-Correos-Pedido-Anonimizada.md	pedido	5	extraccion_pedido.txt + validar DatosExtraidosPedidoDto — nota: Productos usualmente [] (van en adjunto)
TPSC-RE-FU-008-Muestra-Correos-Cotizacion-Anonimizada.md	requisicion	5	extraccion_cotizacion.txt + validar DatosExtraidosCotizacionDto — Urgencia es texto libre
TPSC-RE-FU-008-Muestra-Correos-Otros-Anonimizada.md	otros	4	Casos negativos para clasificacion.txt — sin prompt de extracción



CSharp

```
//Mailbot.Domain/Interfaces/IEventoChannel.cs
//Implementa:Mailbot.Infrastructure/Channels/EventoCorreoChannel.cs
(System.Threading.Channels)
public interface IEventoChannel
{
    ValueTask PublicarAsync(EventoCorreoDto evento, CancellationToken ct = default);
    IAsyncEnumerable<EventoCorreoDto> ConsumirAsync(CancellationToken ct = default);
}
// Mailbot.Domain/Interfaces/ICorreoRepository.cs
// Implementa: Mailbot.Infrastructure/Persistence/Repositories/CorreoRepository.cs
public interface ICorreoRepository
{
    Task<bool> ExisteEventoAsync(string identificadorCorreoGmail, Guid idRegion,
    CancellationToken ct = default);
    Task<Guid> RegistrarEventoAsync(string identificadorCorreoGmail, Guid idRegion, string
    historyId, CancellationToken ct = default);
    Task<Guid> GuardarCorreoAsync(CorreoCompletoDto correo, ResultadoClasificacionDto
    clasificacion, Guid idRegion, CancellationToken ct = default);
    Task MarcarProcesadoAsync(Guid idMailbotEventoCorreo, CancellationToken ct = default);
    Task RegistrarErrorAsync(Guid idMailbotEventoCorreo, string mensajeError,
    CancellationToken ct = default);
}
```

4. Query SQL — Buzón de Cobros

Fuente: documentación de BD del requisito R16A-RE-FU-008 (sección "Bandeja del Buzón de Cobros por Gestor"). Es la base de BuzonCobrosB0 (T5) y de POST /BuzonCobros; los filtros adicionales de QueryInfo (texto, fecha, leído) se agregan como condiciones AND sobre este mismo SELECT.

SQL

```
DECLARE @IdUsuarioCobrador UNIQUEIDENTIFIER;
DECLARE @ClaveCobro VARCHAR(150) = 'cobro';

SELECT
    cr.IdCorreoRecibido,
    cr.Asunto,
    cr.CorreoEmisor,
    cr.FechaRecepcion,
    crc.IdCorreoRecibidoCliente,
    crc.Leido,
    crc.Procesado,
    c.Nombre AS Cliente,
    r.ClaveISO AS Region,
    ffc.IdFCCFolioPagoCliente,
```



```

    ffc.Folio      AS FolioCobro,
    ffc.TotalMailBot
FROM dbo.CorreoRecibidoCliente crc
INNER JOIN dbo.CorreoRecibido cr
    ON crc.IdCorreoRecibido = cr.IdCorreoRecibido
INNER JOIN dbo.catClasificacionCorreoRecibido cat
    ON crc.IdCatClasificacionCorreoRecibido = cat.IdCatClasificacionCorreoRecibido
INNER JOIN dbo.Cliente c
    ON crc.IdCliente = c.IdCliente
INNER JOIN dbo.Region r
    ON cr.IdRegion = r.IdRegion
INNER JOIN dbo.ClienteCarteraCliente ccc
    ON c.IdCliente = ccc.IdCliente AND ccc.Activo = 1
INNER JOIN dbo.ClienteCartera cc
    ON ccc.IdClienteCartera = cc.IdClienteCartera AND cc.Activo = 1
LEFT JOIN dbo.fccFolioPagoCliente ffc
    ON crc.IdCorreoRecibidoCliente = ffc.IdCorreoRecibidoCliente AND ffc.Activo = 1
WHERE cat.Clave          = @ClaveCobro
    AND cc.IdUsuarioCobrador = @IdUsuarioCobrador
    AND crc.Activo          = 1
ORDER BY cr.FechaRecepcion DESC;

```

5. Endpoints internos de Mailbot.Api

Estos endpoints viven en Mailbot.Api (.NET 10), no en ProquifaDotNet.

POST /webhook/gmail/{region}

Propiedad	Descripción
Path	region: string — mex \ per
Headers	Authorization: Bearer {google-push-token} — validado por PubSubTokenValidationMiddleware
Request body	PubSubNotificacionDto — { "message": { "data": "base64", "messageId": "string", "publishTime": "2026-06-17T10:00:00Z" }, "subscription": "string" }
Decodificado de data	{ "emailAddress": "mailbot@proquifa.mx", "historyId": "12345" }
Response	200 OK (siempre que el token sea válido — ver Detalle de idempotencia, Flujo 1); 401 si el token de Google es inválido

GET /health

Propiedad	Descripción
Autenticación	Ninguna — endpoint de infraestructura (probes de balanceador / orquestador)
Response 200	{ "status": "Healthy" \ "Degraded" \ "Unhealthy", "checks": { "sqlServer": "Healthy", "gmailApi": "Healthy", "vertexAI": "Healthy" }, "timestamp": "2026-06-17T10:00:00Z" }

GET /metrics

Formato: JSON simple, no Prometheus — no existe servidor Prometheus ni Grafana en ningún repo del ecosistema (Core.Pqf, ProquifaDotNet, ni el arquetipo microservice-clean-architecture-template, que ya trae Serilog); montar esa infraestructura solo para este endpoint interno no se justifica

Propiedad	Descripción
Autenticación	Interna / red privada — no debe ser pública
Response 200	{ "correosRecibidosHoy": 42, "correosProcesados": 40, "correosConError": 2, "watchExpiraEnSegundos": 345600, "colaChannelTamano": 3 }

Fuera de alcance — POST /reentrenamiento/trigger: se contempló originalmente por nombre heredado de la arquitectura ML.NET, pero se confirmó contra el código legacy real (ConsumidorMailBot/Program.cs) que nunca existió un proceso de reentrenamiento operativo — el modelo solo se entrena una vez, si el archivo .zip no existe en disco; no hay job programado ni acción de negocio que lo dispare. El Agente IA (LLM) tampoco reentrena pesos. Si en el futuro se necesita revisar clasificaciones incorrectas, ya existe MailbotClasificacionLog para ese reporte.

GET /admin/mailbot/eventos

Bitácora consultable de correos en estado NO PROCESADO para el equipo de operaciones — cierra Gap 3 del análisis de rediseño de clasificación.

Propiedad	Descripción
Query params	estado=no-procesado — filtra Procesado = 0 con Error no nulo; pagina (default 1), tamaño (default 50)

Autenticación	Interna / red privada — mismo patrón que /health y /metrics, no expuesta públicamente
Response 200	{ "total": 3, "items": [{ "id": "guid", "identificadorCorreoGmail": "mailbot@proquiifa.mx:12345", "region": "MEX", "error": "GeminiException: timeout", "motivoRechazo": "ErrorTecnico", "intentos": 3, "fechaEvento": "2026-06-25T10:00:00Z", "diasDesdeError": 2 }] }

POST /admin/mailbot/eventos/{id}/retry

Propiedad	Descripción
Path	id: Guid — IdMailbotEventoCorreo
Request body	Ninguno
Autenticación	Interna / red privada
Comportamiento	Resetea Intentos = 0, Error = NULL, MotivoRechazo = NULL en MailbotEventoCorreo; republica EventoCorreoDto al Channel interno (mismo flujo que el push original). El timer de 30 días (RT-20) se reinicia desde la fecha del retry. Si vuelve a fallar: aplica el mismo backoff (3 intentos → DLQ) + alerta Brevo — la alerta no se suprime aunque ya se haya enviado antes; ops debe saber que el retry también falló.
Response	200 OK si se reencoló; 404 si id no existe; 409 Conflict si Procesado = 1 (ya resuelto, sin acción posible)

POST /admin/mailbot/eventos/{id}/descartar

Propiedad	Descripción
Path	id: Guid — IdMailbotEventoCorreo
Request body	{ "notaRechazo": "string" } — descripción obligatoria de por qué se descarta
Autenticación	Interna / red privada
Comportamiento	Establece MotivoRechazo = 'OtroMotivo', NotaRechazo = <nota>, Procesado = 1, FechaProcesado = GETDATE(). El correo deja de aparecer en GET /admin/mailbot/eventos?estado=no-procesado.
Response	204 No Content si se descartó; 404 si id no existe; 409 Conflict si Procesado = 1; 400 si notaRechazo está vacío o nulo



Impacto Técnico

1. Impacto en código existente

Modificaciones en ProquifaDotNet (.NET 4.8):

Archivo	Razón del cambio
GeneradorProcesoMailBotB0.cs	Refactorizar switch: evaluar Clave (no display), activar case "cobro" → ToPagoB0. Bug L3 y L4 del legacy.
catClasificacionCorreoRecibido (BD)	INSERT nuevo registro Clave='cobro', AnalistaDeCuentasPorCobrar=1 (Opción A — Clave='pago' se conserva sin tocar; evidencia: BuzonesPagos.dtsx filtra por Clave='pago' y reutilizarla mezclaría los flujos ETL). UPDATE registro existente Clave='otros' SET AnalistaDeCuentasPorCobrar=1 — permite que Tesorería vea correos no enrutables (RT-18, decisión de Carlos 2026-06-25).
catProceso (BD)	INSERT proceso Cobros con clave cobros.
RegionConfiguracionMailBot (BD)	ALTER TABLE — agregar cols: PubSubTopic VARCHAR(200), WatchHistoryId VARCHAR(50), WatchExpiration DATETIME.

Componentes nuevos en ProquifaDotNet (.NET 4.8):

Archivo	Ubicación	Propósito
ClasificacionCorreoRecibidoConstants.cs	Logic.Pqf.Logistica/L11.MailBot/Models/	Constantes técnicas para switch (Requisición / Pedido / Cobro / Otros) — reemplaza strings hardcoded.
BuzonCobrosB0.cs	Logic.Pqf.Logistica/L11.MailBot/	Lista paginada del Buzón de Cobros (filtro gestor + región). Patrón: ConstruirBuzonCotizacionTransaccionB0.cs existente.
BuzonCobrosB0.Reclasificar.cs	Logic.Pqf.Logistica/L11.MailBot/	Reclasificación manual de correo a otro buzón.
FolioPagoClienteB0.cs (modificación)	Logic.Pqf.Logistica/L11.MailBot/Procesos/Pagos/CorreoRecibidoClienteToPagoB0.cs	No es archivo nuevo. Generación automática de fccFolioPagoCliente al

		clasificar como cobro — se agrega a CorreoRecibidoClienteToPagoB0.cs existente.
FolioPagoClienteB0.Cierre.cs (modificación)	Logic.Pqf.Logistica/L11.MailBot/Procesos/Pagos/CorreoRecibidoClienteToPagoB0.cs	No es archivo nuevo. Cierre/eliminación automática del pendiente por eventos de Validar Cobro — mismo archivo que la fila anterior.
BuzonCobrosController.cs	WebApi.Logistica/Controllers/Procesos/Mailbot/	POST /BuzonCobros + PUT /api/buzon/cobros/{id}/reclasificar + PUT /api/cobros/folio/{id}/cerrar.
BandejaCoordinadorTesoreriaB0.cs	Logic.Pqf.Logistica/L11.MailBot/	Lista paginada de correos de cobro no enrutables (OBS-021 Casos 1 y 2).
BandejaCoordinadorTesoreriaB0.Reclasificar.cs	Logic.Pqf.Logistica/L11.MailBot/	Reclasificación a "Otros" únicamente, con validación de folio activo (decisión D3 — Tarea T36).
BandejaCoordinadorTesoreriaController.cs	WebApi.Logistica/Controllers/Procesos/Mailbot/	POST /BandejaCoordinadorTesoreria + PUT /bandeja/coordinador/{id}/reclasificar (solo rol Coordinador de Tesorería — Tarea T37).
FU-008_CatalogosCobros.sql	Script BD (T1-T3)	INSERT catClasificacionCorreoRecibido Clave='cobro' + INSERT catProceso Cobros + ALTER RegionConfiguracionMailBot + CREATE TABLE MailbotEventoCorreo y MailbotClasificacionLog.



Soluciones nuevas:

Solución	Framework	Propósito	Asignado a
MailbotWorker.sln	.NET 10	Receptor Gmail Push + clasificación IA + persistencia BD	Carlos Ivan Morales Carreon
ProquifaDotNet.LegacyBridge	.NET 10	Infraestructura base de sincronización a Legacy (PConnect). Canal ETL Cobros: Buzón de Cobros.	Carlos Ivan Morales Carreon

Orden de arranque obligatorio: Ejecutar scripts BD (T1-T3 y LB-T1) antes de cualquier tarea de código. T22 (Pub/Sub GCP) y T23 (Gmail watch) son prerequisites del Worker en Staging/Prod.

2. Impacto en modelos por actualización de modelos

ProquifaDotNet — BD ProquifaDotNet

Entidad	Estado	Cambio
catClasificacionCorreoRecibido	Existente	INSERT nuevo registro Clave='cobro', AnalistaDeCuentasPorCobrar=1 (Opción A)
catProceso	Existente	INSERT proceso Cobros con clave cobros
RegionConfiguracionMailBot	Existente	ALTER TABLE — agregar PubSubTopic VARCHAR(200), WatchHistoryId VARCHAR(50), WatchExpiration DATETIME
CorreoRecibido	Sin cambios	IdRegion — segregación MEX/PER ya existe
CorreoRecibidoCliente	Sin cambios	IdCliente nullable — ya soporta Caso 2 de Coordinador
fccFolioPagoCliente	Sin cambios	Campos *MailBot ya existen — se pre-cargan desde datosExtraídos del Agente IA

ClienteCartera	Sin cambios	IdUsuarioCobrador — filtro de visibilidad del buzón
MailbotEventoCorreo	NUEVA	Trazabilidad de notificaciones Pub/Sub e idempotencia (índice UNIQUE en IdentificadorCorreoGmail)
MailbotClasificacionLog	NUEVA	Auditoria de todos los intentos de clasificacion del Agente IA — exitosos y fallidos (RT-21). Columnas: ClasificacionIA (nullable en fallo), Confianza, ModeloVersion, Tokens, ErrorIA NVARCHAR(500) NULL, TipoFallo NVARCHAR(50) NULL
catCobroEstatus	NUEVA — creada por RE-002, no por FU-008	Catálogo del ciclo de vida del cobro: BORRADOR→CAPTURADO→ASOCIADO/SALDO_A_FAVOR→COMPLETADO; CON_INCONSISTENCIA/CANCELADO en cualquier punto. Corregido 2026-06-22 (P17): el cliente trasladó la definición completa de R16A-RE-FU-008_BD.md a R16A-RE-FU-002_BD.md (Cobrador/Cartera) — RE-002 (otro equipo) crea el catálogo, FU-008 solo lo consume vía FK. Tarea T40 eliminada del Plan de FU-008. FU-008 no maneja las transiciones de estado — las ejecutan los sub-requisitos de Validar Cobro (RE-023 a RE-030), fuera de este alcance
fccPagoCliente	Existente + ALTER	Agrega FK IdCatCobroEstatus (default

		BORRADOR) — Tarea T41, prerequisite: entrega externa de RE-002 (ya no T40 propia, ver P17). Backfill de históricos diferido (P18, decisión de Carlos) : el ALTER simple basta por ahora — nada consume el campo todavía; curar el histórico queda como condición previa al go-live de Validar Cobro, con reglas definidas por quien diseñe RE-023 a RE-030
--	--	--

PConnectProquifaDotNet — BD existente, reutilizada por LegacyBridge

Precisión (verificado 2026-06-22 contra BD real): PConnectProquifaDotNet *no es una BD nueva — existe desde 2023-10-02 y ya es la BD intermedia real para el ETL vía SSIS de Cotizaciones y Pedidos* (Cotizaciones, BuzonPedido, y el mecanismo de control Transferencia/EjecucionInterfaz/BitacoraInterface/catEstadoTransferencia, *activo hoy con miles de filas reales*). *Solo las 4 tablas/vista listadas abajo son nuevas, agregadas dentro de esa BD ya existente.* LegacyBridge *no se integra a ese mecanismo legacy — usa su propio esquema SyncControl/SyncJobLog por decisión explícita: es una solución nueva que mejora lo existente, no una extensión del patrón T-SQL legacy*

Objeto	Tipo	Estado	Propósito
SyncControl	Tabla	NUEVA	Estado de cada registro a sincronizar: Pendiente / EnProceso / Completado / Error
SyncJobLog	Tabla	NUEVA	Log por ejecución con snapshot JSON del payload; clasifica errores Transient vs Permanent
AppSettings	Tabla	NUEVA	Configuración en caliente sin recompilar (destinatarios Brevo, maxReintentos, etc.)
vw_SyncPendientes	Vista	NUEVA	Registros SyncControl en estado Pendiente o Error+Transient elegibles para reintento

3. ProquifaDotNet.LegacyBridge — impacto detallado



Propósito: Infraestructura base de sincronización desde ProquifaDotNet hacia el sistema Legacy (PConnect / ProquifaNet 2). FU-008 construye el primer canal ETL (ETLCobros — Buzón de Cobros). Las tareas LB-T1 a LB-T10 son reutilizables por futuros requisitos de R16 que requieran sincronización a Legacy. **Por decisión confirmada, LegacyBridge es exclusivamente Canal ETLCobros por ahora** — Cotizaciones y Pedidos se quedan en su ETL SSIS actual (Transferencia/BuzonCotizacion/BuzonPedido), sin migración planeada; la reutilización de LB-T1-T10 es para requisitos futuros, no un compromiso de absorber lo existente.

Asignado a: Carlos Ivan Morales Carreon (FU-008)

Estructura de proyectos:

Proyecto	Responsabilidad
LegacyBridge.Domain	Entidades: SyncControl, SyncJobLog, AppSettings. Interfaces: IRepository<T>, ISyncJobLog, IExceptionClassifier, IFileSync, ISyncChannelHandler
LegacyBridge.Application	CQRS con MediatR (Commands/Queries genéricos), SyncControlService (gestión de transiciones de estado, elegibilidad de reintento y registro de pendientes), SyncChannelRegistry/ISyncChannelRegistry (despacho genérico por canal), SyncJobDispatcher, DTOs
LegacyBridge.Infrastructure	3 contextos EF Core: ProquifaDotNetDbContext (origen), PConnectDbContext (Legacy), LegacyBridgeDbContext (control). ExceptionClassifier (Transient/Permanent), FileSyncService (MinIO → red Legacy), BrevoNotificationService
LegacyBridge.Worker	Hangfire + SyncJobBase abstracto: flujo pendientes → EnProceso → ejecutar → log → Completado/Error. Cada canal ETL hereda e implementa solo EjecutarSyncAsync. Storage en PConnectProquifaDotNet. Agrega BarridoPendientesJob (recurring, cada 30 min) y extiende AutomaticRetryFailureLogFilter para notificar fallo definitivo tras agotar reintentos (ver subsección P20 abajo)



LegacyBridge.API	GET /sync/estado (consulta por entidad/ID), POST /sync/reintentar (solo si Estado=Error y TipoError != Permanent), POST /sync/registrar (registra un nuevo pendiente, ver subsección P20 abajo). Autenticación vía IdentityServer. Swagger/OpenAPI
LegacyBridge.Testing	Pruebas unitarias: ExceptionClassifier, SyncControlService, SyncJobBase (mocks). Pruebas de integración contra QA — no se ejecutan en CI por commit

Mecanismo de orquestación — decisión confirmada (2026-06-22): *Hangfire, evaluado formalmente contra RabbitMQ, Kafka y otras alternativas en el DAR de Mecanismo de Orquestación LegacyBridge (v1.2). La decisión no es por falta de alternativas reales — Hangfire (TaskSchedulerPqf) y RabbitMQ (Core.Pqf/ProquifaDotNet, proquifa-punchout-backend) están ambos desplegados y activos en este ecosistema hoy. Se confirma Hangfire porque: Canal ETL Cobros es un patrón de polling/batch sobre SyncControl, no de reacción a un evento publicado por otro proceso; el precedente de Hangfire está en el mismo dominio del problema (ETL a Legacy, vía SincronizadorPqfLegacy, ~95-98% de éxito) mientras los precedentes de RabbitMQ resuelven otro tipo de negocio; exige una sola dependencia operativa (el mismo SQL Server de SyncControl) en vez de un broker aparte que mantener y asegurar; y preserva el desacoplamiento entre las tres soluciones en paralelo de este diseño, que un esquema productor/consumidor rompería.*

Mecanismo de detección y disparo del ETL — P20 resuelto (2026-06-24): *el flujo de abajo describe cómo Hangfire procesa una fila SyncControl ya en Pendiente, pero faltaba definir quién la crea. Restricción confirmada con el equipo: toda comunicación entre servicios se hace vía llamadas API — descartado el INSERT directo cross-base y el trigger de BD cross-database. Diseño de dos capas, ambas vía API: (1) push inmediato — MailbotWorker.sln (GenerarPendienteUseCase) llama a POST /sync/registrar justo tras insertar fccFolioPagoCliente (ver Flujo 1, paso 14); (2) barrido de respaldo — BarridoPendientesJob (recurring, cada 30 min) cruza un endpoint de solo lectura nuevo en ProquifaDotNet (GET /api/buzon/cobros/recientes) contra el propio SyncControl, para el caso donde el push nunca llega. El despacho del job se resuelve con SyncChannelRegistry/SyncJobDispatcher en vez de atarse a BuzonCobrosEtlJob por nombre, para que un canal futuro no requiera tocar el contrato. Diseño completo, diagramas de secuencia y modelo de datos propuesto para SyncControl: ver documentos de Arquitectura LegacyBridge Worker (lado receptor) y*

Arquitectura MailbotWorker (lado emisor). Detalle de despliegue en la subsección "Despliegue" más abajo.

Flujo de sincronización (Hangfire):

None

```
POST /sync/registrar (push, MailbotWorker) o BarridoPendientesJob (respaldo)
  → SyncControlService.RegistrarPendiente()
    → INSERT SyncControl (Pendiente) [idempotente: 200 si ya existía]
  → SyncJobDispatcher.ProcesarPendiente()
    → Resolver handler vía SyncChannelRegistry (por NombreEntidad)
    → UPDATE SyncControl SET Estado = EnProceso
    → EjecutarSyncAsync() [implementado por cada canal ETL]
      → Verificar si el registro ya existe en destino (clave natural de origen)
      → Si ya existe: marcar Completado sin reinsertar (idempotencia)
      → Si no existe: INSERT en Legacy (PConnect)
    → INSERT SyncJobLog (snapshot JSON del payload)
    → UPDATE SyncControl SET Estado = Completado
  → Si error Transient (timeout/deadlock):
    → Hangfire reintenta con backoff exponencial
    → UPDATE SyncControl SET Estado = Error, TipoError = Transient
    → Si se agotan los 5 reintentos (Hangfire FailedState):
      → AutomaticRetryFailureLogFilter detecta el FailedState
      → Llama a la notificación de fallo definitivo (mecanismo a definir aparte – ver doc de arquitectura)
  → Si error Permanent (constraint/negocio):
    → BrevoNotificationService.EnviaAlerta() – incluye entidad, IdOrigen, intentos, enlace /reintentar
    → UPDATE SyncControl SET Estado = Error, TipoError = Permanent
    → NO se reintenta automáticamente
```

Idempotencia (verificado contra el patrón legacy real, 2026-06-22): el mecanismo legacy de BuzonPedido/Cotizaciones no usa ningún constraint de BD para evitar duplicados — se apoya solo en disciplina de orquestación (no seleccionar trabajo nuevo con transferencia abierta + reanudar en vez de reprocesar). LegacyBridge mejora esto agregando un constraint único real sobre la clave natural de origen (ej. IdCorreoRecibidoCliente) en cada tabla Legacy nueva (BuzonCobros para Canal ETL Cobros), además del chequeo en código del paso anterior — defensa en profundidad para el caso donde el job muere justo después del INSERT a Legacy pero antes de marcar Completado.

Contratos de los endpoints nuevos (P20):

Endpoint	Método	Body / Query	Respuesta	Quién lo llama
/sync/registrar	POST	{ NombreEntidad,	201 nuevo / 200 idempotente / 400 entidad no	MailbotWorker.s ln (push) y

		IdentificadorRegistro }	registrada en SyncChannelRegistry	LegacyBridge.Worker (barrido)
/api/buzon/cobros/recientes	GET	?desde=<timestampt>	Lista de fccFolioPagoCliente cobros activos creados desde esa fecha	LegacyBridge.Worker (barrido) — nuevo endpoint de solo lectura en BuzonCobrosController (ProquifaDotNet)

Canal ETL Cobros — Buzón de Cobros (especifico FU-008):

Componente	Proyecto	Descripción
IEtlBuzonCobrosLegacyService	LegacyBridge.Application	Interfaz del canal ETL para datos del Buzón de Cobros hacia PConnect
EtlBuzonCobrosLegacyServiceStub	LegacyBridge.Infraestructure	Stub para pruebas mientras el detalle final de DoctosR no está confirmado (ETLCobros-T11)
EtlBuzonCobrosLegacyService	LegacyBridge.Infraestructure	Implementación definitiva — reemplaza el stub una vez confirmado el detalle (ETLCobros-T12)
BuzonCobrosEtlJob	LegacyBridge.Worker	Hereda SyncJobBase, implementa EjecutarSyncAsync para el canal ETL Cobros

Canal ETL Legacy de Cobros — resuelto para diseño: confirmado contra PConnectProquifaDotNet real que no existe BuzonCobros — solo BuzonPagos (vacía, 0 filas) y BuzonPedido (292 filas reales). El molde de tabla a seguir, verificado contra el esquema real: IdCorreoRecibidoClienteReferencia/IdCorreoRecibidoCliente (FK al correo en ProquifaDotNet), CorreoEmisor, DoctoR (int, **sin FK** — no es un código de tipo de documento como se asumía, son valores grandes secuenciales: el folio/documento que asigna el sistema legacy real al sincronizar), FechaRegistro/FechaTramitacion, Bitacora, ArchivoSincronizado, Actualizado, RegistroCompleto, Insertado. **Se confirmó además el linked server LegacyAux (→ otra instancia SQL Server, RYNL015) como el verdadero destino del valor de DoctoR — coincide con la referencia histórica a DoctosR. No fue posible conectar directo a esa instancia (incompatibilidad de protocolo TLS con**

herramientas modernas) para confirmar el esquema exacto de DoctosR ni el valor a usar para "Cobro" — eso sigue siendo el alcance específico de ETLCobros-T10, ya contemplado en el plan (ver TPSC-RE-FU-008-Plan-v1.md). El esquema ya alcanza para diseñar BuzonCobros y arrancar LB-T1/T2 con el stub. Por la decisión de idempotencia (ver nota arriba, LB-P4), BuzonCobros debe llevar además un índice único sobre IdCorreoRecibidoCliente (la clave natural de origen) — algo que BuzonPedido/BuzonPagos nunca tuvieron, posible aquí porque la tabla es nueva.

Despliegue (LegacyBridge.API / LegacyBridge.Worker): misma convención del arquetipo microservice-clean-architecture-template ya usada para MailbotWorker.sln (ver "Arquitectura de la solución MailbotWorker.sln" arriba) — sin desviación.

Proyecto	Cómo se publica	Cómo se instala
LegacyBridge.Worker	win-x64, framework-dependent	Windows Service vía sc.exe create — proceso independiente, no requiere IIS
LegacyBridge.API	win-x64, framework-dependent	Sitio IIS (requiere ASP.NET Core Module - ANCM) — mismo IIS donde ya corre ProquifaDotNet (.NET 4.8); no es infraestructura nueva, es un sitio adicional

Ambos comparten las capas Application/Infraestructure/Domain como referencias de proyecto — son procesos independientes, sin llamada de red entre ellos para su propio funcionamiento interno.

Diseño — Buzones de Cotizaciones y Pedidos (T25-T32)

Mismo patrón que el Buzón de Cobros, con dos diferencias clave marcadas explícitamente abajo. ConstruirBuzonCotizacionTransaccionB0.cs ya existe en el proyecto como buzón legacy — T25-T28 adaptan ese BO existente, no crean uno desde cero. El Buzón de Pedidos (T29-T32) es nuevo siguiendo el mismo patrón.

Nomenclatura — "Cotización" es la clave legacy requisicion: confirmado directamente contra código real en producción. ConstruirBuzonCotizacionTransaccionB0.cs línea 35: context.catClasificacionCorreoRecibido.FirstOrDefault(x => x.Clave == "requisicion" && x.Activo) — "Cotización" es la clave requisicion, no hay clave nueva que insertar (a diferencia de cobro). El Agente IA debe clasificar usando requisicion, y T25-T28 deben filtrar/usar esa misma clave.

Diferencias frente al Buzón de Cobros

Aspecto	Buzón de Cobros	Buzón de Cotizaciones	Buzón de Pedidos
Filtro de visibilidad	ClienteCartera.IdUsuarioCobrador (RT-07)	ClienteCartera.IdUsuarioESAC — mismo patrón, campo distinto. Reutiliza ObtenerIDUsuarioESACPorCliente() ya existente en ClienteCarteraClienteBO.Extensions.cs	Igual que Cotizaciones
Pendiente generado al clasificar	fccFolioPagoCliente	cotCotizacion — confirmado contra código real (GeneradorProcesoMailBotTransaccionBO.cs línea 102, ConstruirBuzonCotizacionTransaccionBO.cs): se crea vía cotCotizacionFactoryTransaction.Fabricar()	pcPromesaDeCompra — confirmado contra código real (GeneradorProcesoMailBotTransaccionBO.cs línea 78)
Estructura del pendiente	IdCorreoRecibidoCliente (FK) + Activo (bit)	Mismo patrón — cotCotizacion tiene IdCorreoRecibidoCliente (FK) y Activo (bit), verificado contra esquema real	Mismo patrón — pcPromesaDeCompra tiene IdCorreoRecibidoCliente (FK) y Activo (bit), verificado contra esquema real
Cierre/eliminación automática	Eventos de Validar Cobro (Reglas 6/7)	Análogo al de Cobros: cotCotizacion.Activo = 0 cuando se cierra/cancela la cotización en su propio módulo (fuera de alcance FU-008, igual que Validar Cobro lo está)	Análogo: pcPromesaDeCompra.Activo = 0 cuando se cierra/cancela en su propio módulo

Bloqueo al reclasificar (D1)	Sí, si fccFolioPagoCliente .Activo = 1	Aplica igual — bloquear con 409 Conflict si cotCotizacion.Activo = 1 para ese IdCorreoRecibidoCliente (mismo patrón D1, T26)	Aplica igual — bloquear si pcPromesaDeCompra.Activo = 1 (mismo patrón D1, T30)
------------------------------	--	--	--

T33 (GenerarPendienteUseCase) *sin ambigüedad:* para clasificacion == 'requisicion' debe crear/vincular cotCotizacion (reutilizando cotCotizacionFactoryTransaction, igual que el legacy); para clasificacion == 'pedido' debe crear/vincular pcPromesaDeCompra. No se inventan entidades nuevas — son tablas y factories que ya existen y están en uso activo en el sistema.

Flujo — Consulta y reclasificación (ambos buzones, mismo patrón)

- El correo se clasifica como pedido o requisicion en el Flujo 1 — mismo mecanismo que Cobros, incluyendo el paso 14 (genera pcPromesaDeCompra/cotCotizacion según el caso — T33).
- El ESAC consulta POST /BuzonCotizaciones o POST /BuzonPedidos — mismo patrón que POST /BuzonCobros (decisión C6: POST, no GET).
- BuzonCotizacionesB0 / BuzonPedidosB0 filtra por IdUsuarioESAC del token JWT y IdRegion.
- Reclasificación manual vía PUT /api/buzon/cotizaciones/{id}/reclasificar o PUT /api/buzon/pedidos/{id}/reclasificar — mismo contrato y mismo bloqueo 409 (D1) que PUT /api/buzon/cobros/{id}/reclasificar, validando cotCotizacion.Activo/pcPromesaDeCompra.Activo según el buzón.

Contratos de endpoints

Endpoint	Request	Response
POST /BuzonCotizaciones	QueryInfo	QueryResult<BuzonCotizacionesDetalleDto>
PUT /api/buzon/cotizaciones/{id}/reclasificar	GMReclasificarCorreo	204 / 404 / 400
POST /BuzonPedidos	QueryInfo	QueryResult<BuzonPedidosDetalleDto>
PUT /api/buzon/pedidos/{id}/reclasificar	GMReclasificarCorreo	204 / 404 / 400

BuzonCotizacionesDetalleDto / BuzonPedidosDetalleDto (mismo shape para ambos):

Campo	Tipo	Nulo
IdCorreoRecibido	Guid	No
Asunto	string	No
CorreoEmisor	string	No
FechaRecepcion	DateTime	No
Cliente	string	No
Region	string	No
Leido	bool	No
Procesado	bool	No

Sin columnas equivalentes a FolioCobro/TotalMailBot — no hay folio monetario que mostrar en estos dos buzones.

Corte de producción del Mailbot legacy (decisión D5)

*El Agente IA de MailbotWorker.sln clasifica los 4 tipos (cobro/pedido/cotizacion/otros) en un mismo paso (Flujo 1, paso 11) — no hay un worker independiente por tipo que se pueda activar por separado. Por eso el corte del ConsumidorMailBot legacy **no puede ser escalonado por buzón**: es un corte único y simultáneo para los 4 tipos, en el mismo despliegue donde MailbotWorker.sln entra en producción (T20+T21+T22+T23+T24 completos).*

- Los correos de Pedidos/Cotizaciones empiezan a clasificarse y persistirse correctamente desde el momento del corte, **incluso si T25-T28/T29-T32 todavía no han terminado** — no hay pérdida de datos durante esa ventana, solo puede tardar más la disponibilidad del Buzón (BO + endpoint + UI) para que el ESAC lo consulte.
- Riesgo evitado: si el legacy siguiera corriendo en paralelo para Pedidos/Cotizaciones mientras MailbotWorker.sln ya está activo para Cobros, ambos sistemas leerían el mismo INBOX de Gmail simultáneamente — clasificación y persistencia duplicada del mismo correo.
- Tarea nueva: **T38** — apagar ConsumidorMailBot (Task Scheduler) en el mismo despliegue (Plan de implementación, Ola 6).

Manejo de Errores y Excepciones

Escenario	Comportamiento esperado
-----------	-------------------------



Token de Google inválido en push Pub/Sub	PubSubTokenValidationMiddleware rechaza con 401. Pub/Sub reintenta — si persiste, el mensaje va a DLQ mailbot-dlq tras agotar reintentos.
Error 500 / timeout en Mailbot.Api al procesar push	Pub/Sub reintenta hasta 5 veces (Ack deadline: 60s). Tras 5 fallos → DLQ. MailbotEventoCorreo.Error se actualiza.
Error en ProcesarCorreoUseCase (Worker)	Backoff exponencial: 5s → 10s → 20s. Tras 3 intentos → NACK → DLQ. UPDATE MailbotEventoCorreo SET Error = mensaje. Brevo envía alerta al equipo de operaciones.
Error de clasificación del Agente IA (timeout / rate limit)	El Worker aplica backoff y reintenta. Si el error es transitorio se recupera. Si es permanente → DLQ + alerta Brevo.
Correo recibido dos veces (reintento Pub/Sub)	MailbotEventoCorreo.IdentificadorCorreoGmail tiene índice UNIQUE — el segundo INSERT falla silenciosamente. No se procesa el correo duplicado.
Watch de Gmail expirado	GmailWatchRenovacionWorker renueva automáticamente cada 6 días. Si falla la renovación → alerta Brevo.
Correo sin cliente identificado (Caso 2)	Se crea CorreoRecibidoCliente con IdCliente = NULL. El correo aparece en la Bandeja del Coordinador de Tesorería en lugar de la bandeja de un Gestor.
Cliente sin Cobrador asignado (Caso 1)	El correo se crea normalmente pero queda fuera de la query del Gestor (filtro IdUsuarioCobrador). Aparece en la Bandeja del Coordinador.
Error al guardar adjunto en MinIO	Log en MailbotClasificacionLog.Error. El registro del correo en BD se persiste de todas formas — el adjunto no bloquea el flujo principal.
Gestor intenta reclasificar un correo con fccFolioPagoCliente.Activo = 1	BuzonCobrosB0.Reclasificar responde 409 Conflict sin modificar la clasificación. El Gestor debe cerrar o marcar inconsistencia en Validar Cobro primero (decisión D1, Opción B).
Coordinador intenta reclasificar a un destino distinto de "Otros"	BandejaCoordenadorTesoreriaB0.Reclasificar rechaza la solicitud — el único destino válido desde esta bandeja es Otros (decisión D3, Opción B, RT-12).
Coordinador intenta reclasificar un correo con fccFolioPagoCliente.Activo = 1	BandejaCoordenadorTesoreriaB0.Reclasificar responde 409 Conflict, misma regla que el Gestor (RT-13).



Equipo de operaciones hace retry manual desde endpoint admin	POST /admin/mailbot/eventos/{id}/retry resetea Intentos = 0, Error = NULL, MotivoRechazo = NULL y republica al Channel. Si vuelve a fallar: mismo backoff + alerta Brevo (no se suprime). Timer de 30 días se reinicia desde la fecha del retry.
Equipo de operaciones descarta correo NO PROCESADO con motivo	POST /admin/mailbot/eventos/{id}/descartar establece MotivoRechazo = OtroMotivo, NotaRechazo = <texto>, Procesado = 1. El correo desaparece del listado de NO PROCESADO.
Agente IA falla antes de devolver resultado (timeout / parse-error / rate-limit)	INSERT en MailbotClasificacionLog con ClasificacionIA = NULL, ErrorIA = <mensaje>, TipoFallo = <categoria> (RT-21). Luego aplica el backoff normal (paso 17). Permite auditar correos que nunca llegaron a clasificarse sin revisar logs de aplicacion.

Estrategia de Pruebas (Diseño de las pruebas)

1. Pruebas funcionales (Criterios de Aceptación en DEV)

- Verificar que un correo de cobro recibido en Gmail INBOX MEX aparece en el Buzón de Cobros del Gestor asignado dentro de los 5 segundos siguientes a la recepción.
- Verificar que al clasificar como cobro se genera automáticamente un registro fccFolioPagoCliente con Activo = 1 y los campos *MailBot pre-cargados.
- Verificar que el Gestor del contexto MEX no ve correos del contexto PER en su Buzón de Cobros.
- Verificar que al reclasificar un correo como "Cotización", desaparece del Buzón de Cobros y aparece en el Buzón de Cotizaciones.
- Verificar que al vincular un cobro a proforma/factura en Validar Cobro, el pendiente desaparece del Buzón de Cobros.
- Verificar que un correo cuyo remitente no está dado de alta como cliente aparece en la Bandeja del Coordinador de Tesorería (Caso 2).
- Verificar que al asignar Cobrador a un cliente en FU-002, los correos del Coordinador pasan automáticamente a la bandeja del Gestor sin acción adicional.
- Verificar que reclasificar un correo con fccFolioPagoCliente.Activo = 1 responde 409 Conflict y no modifica CorreoRecibidoCliente.IdCatClasificacionCorreoRecibido.
- Verificar que, tras cerrar el folio en Validar Cobro (vinculación o inconsistencia), el mismo correo sí se puede reclasificar sin error.
- Verificar que el Coordinador de Tesorería puede reclasificar un correo de su bandeja a "Otros" y que el ESAC lo ve disponible para eliminar desde ahí.
- Verificar que el Coordinador no puede reclasificar a Cobros, Pedidos ni Cotizaciones desde su bandeja.



Verificar que el Coordinador no puede reclasificar un correo con folio activo (409 Conflict), igual que el Gestor.

2. Pruebas técnicas (unitarias e integración)

a. Unitarias

- **ProcesarCorreoUseCaseTests:** verificar que ante una clasificación 'cobro' se invoca `GenerarPendienteUseCase`; que ante 'pedido' no se llama; que errores del Agente IA propagan correctamente.
- **ClasificadorAgenteTests:** verificar que `GeminiClasificadorAgente` construye el prompt correcto usando `PromptBuilder` y que el resultado deserializa correctamente el JSON de clasificación.
- **WebhookGmailEndpointTests:** verificar que un token inválido retorna 401; que un payload válido retorna 200 OK y publica en el Channel; que el INSERT en `MailbotEventoCorreo` ocurre antes de publicar en el Channel.
- **BuzonCobrosB0.Tests:** verificar que la query filtra correctamente por `IdUsuarioCobrador` e `IdRegion`; que correos con `Activo = 0` no aparecen.
- **FolioPagoClienteB0.Tests:** verificar que el INSERT de `fccFolioPagoCliente` pre-carga los campos `*MailBot` desde el DTO de extracción; que el cierre actualiza `Activo = 0`.

b. Pruebas de integración

- **GmailWatchServiceTests:** verificar que `watch.create` registra correctamente en `RegionConfiguracionMailBot.WatchExpiration` y que `GmailWatchRenovacionWorker` renueva antes del vencimiento.
- **ProquifaDbContextTests:** verificar que `MailbotEventoCorreo` con `IdentificadorCorreoGmail` duplicado lanza excepción de constraint UNIQUE; que `MailbotClasificacionLog` registra correctamente la versión del modelo y tokens.
- **ProcesarCorreoUseCase.IntegrationTests:** probar el flujo completo con Gmail API en modo sandbox, Agente IA con mock y SQL Server en QA.

3. Casos críticos

- **Idempotencia ante reintento de Pub/Sub:** Simular que Pub/Sub entrega el mismo push dos veces. Verificar que `MailbotEventoCorreo.IdentificadorCorreoGmail` previene el procesamiento duplicado y que el correo no aparece dos veces en el Buzón.
- **Backoff y DLQ:** Simular un error de clasificación persistente. Verificar que el Worker aplica backoff 5s → 10s → 20s, publica en DLQ `mailbot-dlq`, actualiza `MailbotEventoCorreo.Error` y envía alerta Brevo.
- **Watch de Gmail expirado:** Detener `GmailWatchRenovacionWorker` y esperar vencimiento del watch. Verificar que los correos dejan de recibirse y que la alerta de Brevo se activa.
- **Correo con adjunto grande (>10 MB):** Verificar que el guardado en MinIO no bloquea el flujo principal y que el fallo de adjunto no cancela el registro del correo en BD.



- **Corte de conexión a BD durante INSERT:** Verificar que el Worker aplica backoff correctamente y que no se genera un `fccFolioPagoCliente` huérfano sin `CorreoRecibidoCliente`.



Control de versiones

Versión	Fecha	Autor	Tipo de Cambio	Descripción del cambio	Aprobó
1.0	26 jun 2026	Carlos Iván ...	Creación ▾	Creación del documento.	Juan Davi...

